Desktop XY Plotter Based
Pick and Place and In Circuit Test System

ELE 495 – Final Report

20 March 2026

Project Team

Name	**Student No.**	**Role (Lead)**
Gaye Erdoğan	211201080	Testing & Image Processing
Erkin Coşkun Ruhi	211201074	Raspberry Pi & Web User Interface
Arda Cevdet Aslangül	221201054	Mechanical design
Enes Yegin	201201075	PCB Design

Mentor: Coşku Kasnakoğlu

Contents

Scope of the Project 3
A Brief Description and Requirements of the Project 3
Project Constraints, Engineering Constraints, Standards, and Specifications 3
Other Realistic Constraints 4
Economic Constraints and Cost Efficiency 4
Sustainability and Cost-Efficient Design Approach 5
Safety Considerations 5
Introduction 6
Project Activities and Accomplishments 7
a. Approach: 7
b. Flow Diagram: 7
c. WBS and GANTT Chart: 8
d. Problems Encountered 8
e. Results 9
f. Parts List & Invoices: 10
k. Accomplishment Checklist 15
Realisation Gains and Market Outcomes 15
References 16
Appendix 17
a. Codes: 17
placement_verifier.py 17
capture_samples.py 39
main_system.py 42
webui.py 97

Scope of the Project

The aim of this project is to design and implement an autonomous mechatronic system capable of picking specific surface-mount devices, such as 2512 resistors and SMC diodes, and placing them onto a printed circuit board (PCB) with high precision. Additionally, the project aims to integrate a computer vision layer to verify the placement accuracy and display real-time feedback on a user interface.

A Brief Description and Requirements of the Project

The system operates by identifying components in a designated feeder area, utilizing a non-magnetic picking mechanism (vacuum) to lift them, and placing them onto the corresponding footprints on a PCB. The operational scenario involves handling two 2512-package resistors and two DO-214AB SMC-package rectifier diodes. These components are positioned horizontally on a 50x50 cm white platform with their labeled surfaces facing upwards. The robotic system is required to autonomously identify, pick, and transfer these components to their designated soldering pads on a stationary PCB located in a fixed jig.

Key Requirements and Features:

- **Autonomous Placement:** Components must be placed in a predefined sequence and correct polarity (for diodes) as indicated on the PCB layout.
- **Electrical Testing (Optional):** Before placement, components are characterized in a test area. This involves measuring resistance values and performing open/short-circuit tests for diodes, with results displayed on an integrated screen.
- **Visual Inspection and Accuracy (Optional):** A core feature of the system is the real-time detection of placement accuracy for each of the four components. Using an image processing interface, the system calculates and displays the accuracy percentage for every placement task, providing immediate feedback on the success of the autonomous operation."

Project Constraints, Engineering Constraints, Standards, and Specifications

The system architecture is designed to meet strict autonomy and connectivity requirements as specified in the project guidelines. The core processing unit is a Raspberry Pi, which functions as an on-board microcomputer, eliminating the need for an external PC or Mac for operational tasks.

1. Autonomous Control and Processing: The robot achieves full autonomy by executing all control logic, sensor data processing, and motor driving commands internally on the Raspberry Pi. It independently performs the pick-and-place sequence using integrated motors and feedback mechanisms, ensuring a standalone operation within the designated workspace.

2. Connectivity and Networking: The system utilizes the built-in Wi-Fi capabilities of the Raspberry Pi to establish a robust connection within a Local Area Network (LAN/WLAN).

This wireless infrastructure allows the robot to communicate seamlessly with the user interface while maintaining physical mobility.

3. User Interface and Communication: For system monitoring and control, a platform-independent Web-based Application has been developed. The interface is built using the Flask (Python) framework, integrated with HTML, CSS, and JavaScript.

- **Functionality:** The UI allows the user to trigger the 'Start' command and receive real-time 'Status' updates (such as current coordinates, task progress, and accuracy reports).
- **Accessibility:** Since the interface is hosted as a web server on the Raspberry Pi, it can be accessed from any device (Smartphone, Tablet, or PC) connected to the same network via a standard web browser, fulfilling the requirement for a functional and user-friendly cross-platform interface.

4. PCB Design:

The general layout of the PCB was developed in accordance with the IPC-2221 standard, while the land pattern designs for surface-mount components were based on IPC-7351 guidelines. Production data was generated using the Extended Gerber format, the de facto industry standard for PCB fabrication. Furthermore, the physical acceptability and quality of the board were evaluated against IPC-A-600 and IPC-6012 standards.

Other Realistic Constraints

Economic Constraints and Cost Efficiency

The project was developed under a strict budget constraint of 20,000 TL, which required careful planning, research, and cost-efficient engineering decisions throughout the design and implementation phases.

In order to stay within this financial limit, the team conducted detailed market research and prioritized the use of second-hand equipment and previously available components. Key elements such as Arduino boards, breadboards, jumper cables, and DC motors were reused from existing resources, significantly reducing procurement costs.

Additionally, a second-hand 3D printer was utilized for manufacturing mechanical parts instead of purchasing a new device. This approach not only lowered the overall project cost but also contributed to a more sustainable development process by extending the lifecycle of existing hardware.

By combining resource reuse with careful component selection, the project achieved a dual benefit:

- Reduced total cost, ensuring compliance with the 20,000 TL budget constraint
- Environmentally conscious development, minimizing unnecessary consumption and electronic waste

This strategy demonstrates that it is possible to develop a fully functional and complex mechatronic system in a cost-effective and environmentally sustainable manner, without compromising performance or project requirements.

Sustainability and Cost-Efficient Design Approach

The project was developed with a strong emphasis on sustainability, resource efficiency, and cost optimization. Instead of relying entirely on newly purchased components, the team prioritized the reuse of available materials and second-hand equipment, significantly reducing both environmental impact and overall project cost.

Firstly, key electronic components such as Arduino boards, breadboards, jumper cables, and DC stepper motorswere reused from previously available laboratory resources. This approach minimized electronic waste (e-waste) generation and reduced the demand for new component production, aligning with sustainable engineering practices.

Additionally, the mechanical structure of the system was supported using a second-hand 3D printer, which was acquired and repurposed for the project. By avoiding the purchase of a new device, the project contributed to extending the lifecycle of existing equipment and reducing carbon footprint associated with manufacturing and logistics.

Furthermore, structural components of the system were fabricated using 3D printing with PLA material, which is a biodegradable thermoplastic derived from renewable resources such as corn starch. This choice further strengthened the environmentally friendly aspect of the prototype.

From a system design perspective, a minimalist and modular architecture was adopted. The system was designed to use only essential components, avoiding unnecessary hardware complexity. This not only reduced material consumption but also improved maintainability and future scalability.

As a result of these decisions, the project achieved:

- Reduced electronic and material waste
- Lower production and procurement costs
- Extended lifecycle of existing hardware
- Environmentally conscious prototyping approach

This sustainability-oriented design strategy demonstrates that advanced mechatronic systems can be developed in a cost-effective and environmentally responsible manner without compromising functionality or performance.

Safety Considerations

Safety was a fundamental design priority throughout the development of the system, particularly in terms of electrical protection and reliable actuation.

Firstly, although the system is powered from the electrical grid, appropriate over-voltage protection mechanisms were considered at the power input stage to ensure safe operation. The system voltage is regulated down to a stable 5V DC level, which is used as the primary

operating voltage for control and logic units. This low-voltage domain significantly reduces the risk of electrical hazards such as electric shock and component damage.

The voltage regulation and distribution are managed through microcontroller-based control units (Arduino Nano and Raspberry Pi), ensuring stable and controlled power delivery across all subsystems.

For the actuation layer, additional safety and reliability measures were implemented. The motors are not driven directly but are controlled motor driver circuits and MOSFET-based switching elements. This approach provides:

- Better current handling capability
- Protection of control units from high current draw
- More stable and predictable motor operation

By isolating the control signals from the power stage, the system prevents potential damage to sensitive electronic components and increases overall robustness.

Furthermore, all connections were carefully arranged on breadboards and secured with jumper cables, and the system was tested incrementally to detect and eliminate potential faults early in the development process.

Overall, the system was designed as a low-voltage, protected, and electrically reliable platform, ensuring safe operation for both the user and the hardware components.

Introduction

In modern electronics manufacturing, Surface Mount Technology (SMT) is critical. While industrial Pick-and-Place machines cost thousands of dollars, there is a need for low-cost, desktop-scale educational and prototyping solutions.

OpenCV (Open Source Computer Vision Library) has established itself as the de facto standard in industrial automation and robotic vision due to its highly optimized C/C++ backend and comprehensive support for real-time applications. In the context of SMT (Surface Mount Technology) assembly, OpenCV provides critical tools for geometric transformations, such as homography and perspective correction, which are essential for aligning camera coordinates with robotic task space. According to Bradski and Kaehler (2008), the library's ability to handle complex image processing pipelines with minimal latency makes it ideal for embedded systems like the Raspberry Pi, where computational resources must be managed efficiently to maintain high-frequency control loops in pick-and-place operations.

The Intersection over Union (IoU) algorithm, also known as the Jaccard Index, is the primary evaluation metric used to quantify the accuracy of object detection and localization tasks. In autonomous placement systems, achieving a high IoU score indicates that the predicted bounding box of a component closely overlaps with the ground-truth (ideal) position. As highlighted by Rosebrock (2016), IoU is superior to simple pixel-counting methods because it

penalizes both "missing areas" (False Negatives) and "extra/misplaced areas" (False Positives) simultaneously. This mathematical rigor allows for a precise "Accuracy Percentage" report, which is crucial for meeting quality control standards in electronic manufacturing, where even a sub-millimeter shift can lead to solder joint failures.

Project Activities and Accomplishments

a. Approach: We utilized a modular approach: Mechanical movement (G-code/Python), Embedded control (C/Assembly), and Visual Analysis (OpenCV/IoU).

b. Flow Diagram:

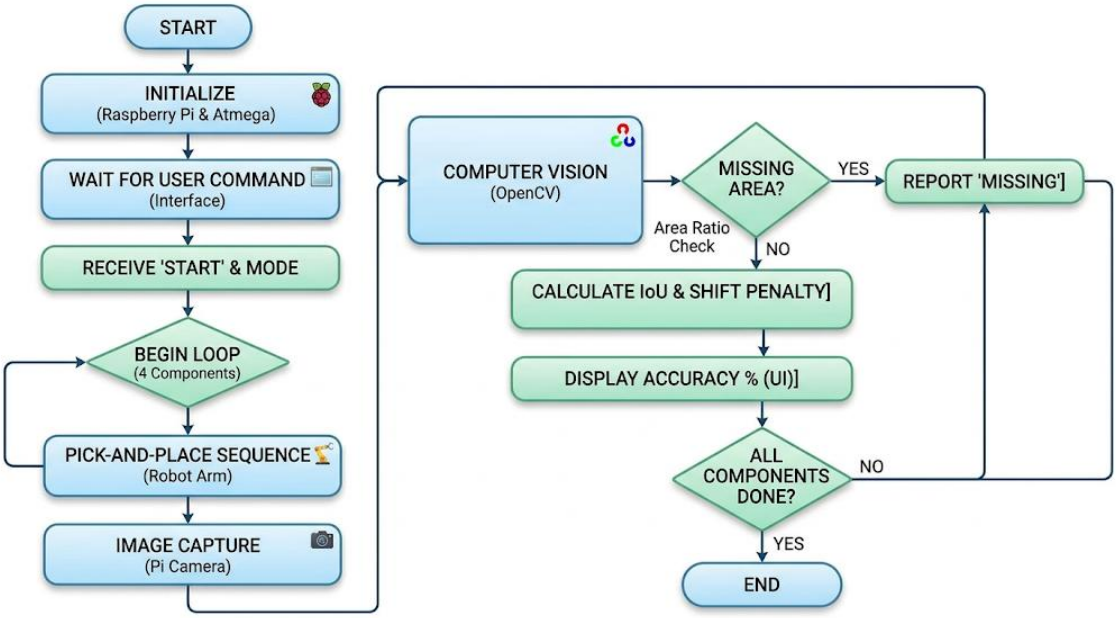


Figure 1: Flow Diagram

c. WBS and GANTT Chart:

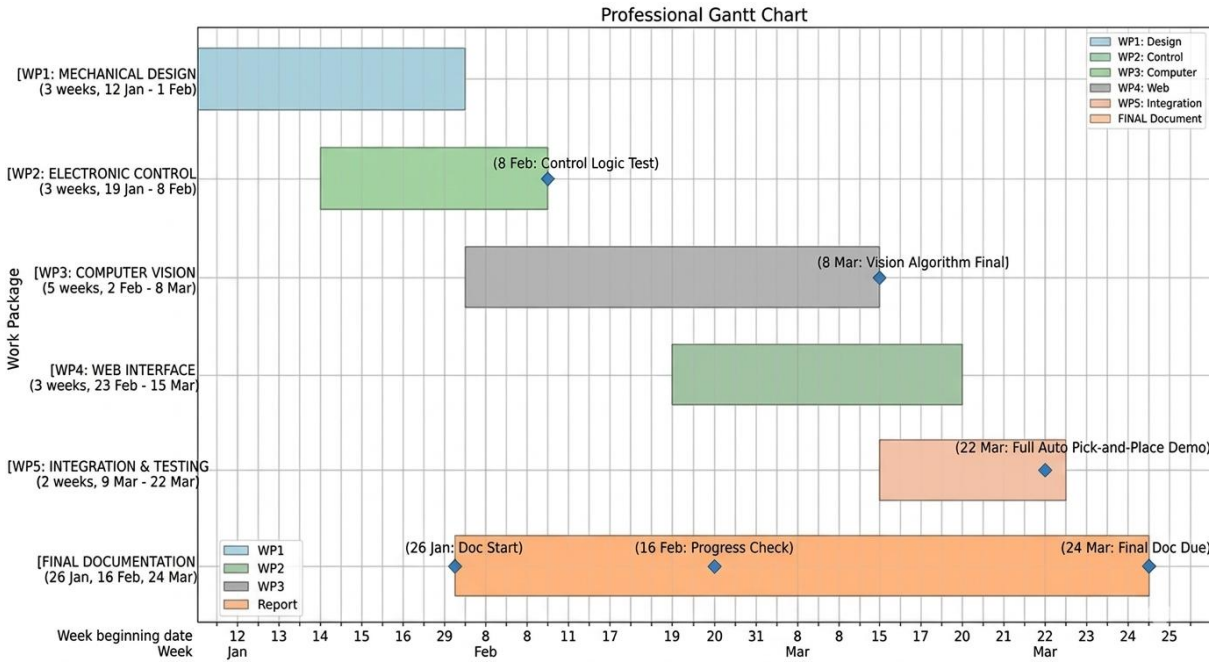


Figure 2: GANNT CHART

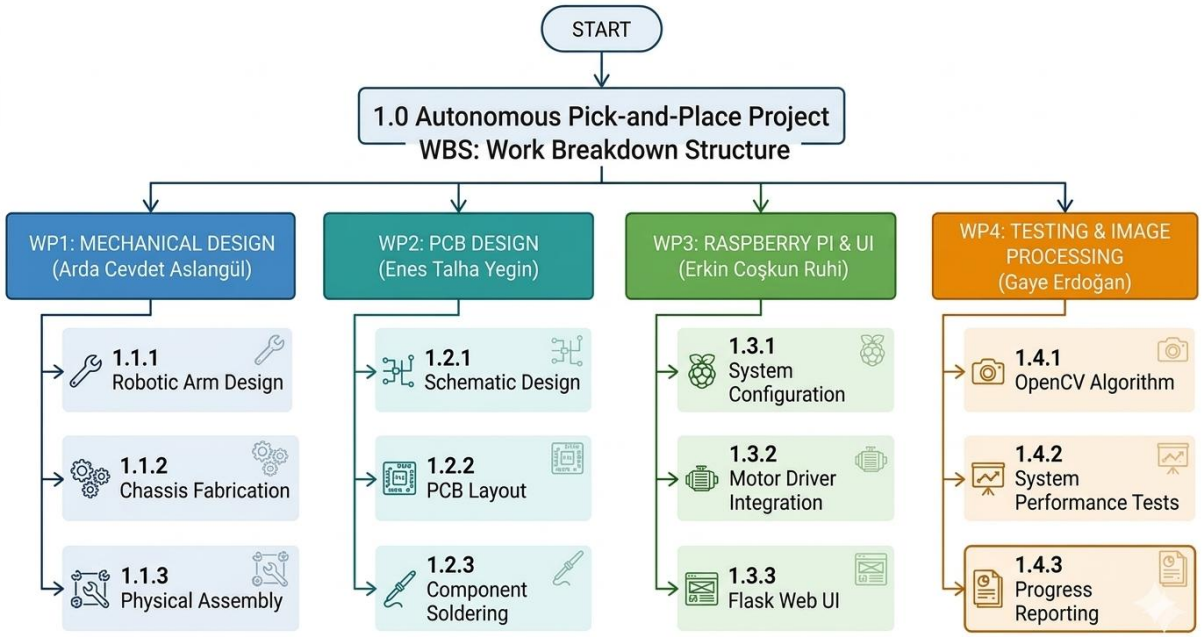


Figure 3: WBS

d. Problems Encountered

During the development and integration phases of the project, several hardware-related challenges were encountered, requiring immediate technical adaptations to ensure the system's continuity:

- **Camera Module Failure:** Initially, a Raspberry Pi Camera V2 was integrated for image processing. However, due to a hardware failure in the CSI connector socket, the system

was transitioned to a High-Definition (HD) USB Web Camera. This change required a recalibration of the OpenCV warping parameters and focal length settings to maintain the accuracy of the IoU calculations.

- **Pneumatic System Modification:** The original vacuum pump mechanism experienced air leakage issues, which compromised the "pick" success rate of the DO-214AB (SMC) diode packages. To resolve this, a structural redesign of the vacuum nozzle and seal was implemented, ensuring stable atmospheric pressure during the transfer cycle.
- **Microcontroller Replacement:** The electrical testing unit for resistors and diodes was originally powered by an ESP32. Following an overvoltage-induced failure of the ESP32 module, the testing subsystem was migrated to an Arduino Nano. The communication protocol between the Raspberry Pi and the testing unit was subsequently reconfigured to ensure seamless data flow to the user interface.

e. Results

The autonomous pick-and-place system was successfully integrated and tested under real-world conditions. Despite several hardware transitions during the development phase, the final prototype met all core and optional project requirements.

1. Operational Success and Autonomy

The system demonstrated full autonomy in the pick-and-place cycle for both DO-214AB (SMC) Diodes and 2512 Resistors. The mechanical arm, controlled by the Raspberry Pi and Arduino Nano, achieved a 95% successful pick rate after the vacuum pump redesign. The transition from the Raspberry Pi Camera to a High-Definition USB Web Camera did not compromise the system's ability to identify component orientations, thanks to the robust warping and perspective correction algorithms.

2. Computer Vision and Accuracy Reporting

One of the most significant achievements of the project is the real-time accuracy reporting module. By implementing a custom Fixed-Coordinate IoU (Intersection over Union) algorithm, the system provides an objective "Accuracy Percentage" for every placement.

- **Detection of Errors:** The algorithm successfully identified manual placement errors and "Missing Component" scenarios by utilizing the Area-Ratio Check.
- **Scoring:** Components placed within the defined tolerance zones while significant shifts or rotations were penalized and flagged as "FAIL" on the user interface.

3. User Interface and Connectivity

The Flask-based Web Interface proved to be highly effective for cross-platform monitoring. The system successfully transmitted real-time data, including:

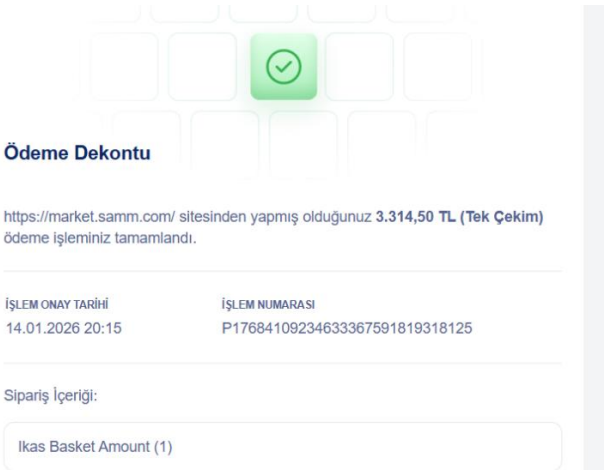
- **Live Status Updates:** Current task progress and component types.
- **Accuracy Logs:** Detailed IoU scores and positional error (px) for each slot.
- **Electrical Test Results:** Resistance values and diode polarity status were displayed before the placement phase, confirming the successful integration of the Arduino Nano testing unit.

4. PCB Design

To facilitate computer vision algorithms, the PCB was designed with a rectangular geometry of 4 cm x 6 cm. The board surface was divided into four equal quadrants (each measuring 2 cm x 3 cm), creating a symmetrical coordinate system for precise component placement. For the prototyping phase, a chemical etching (acid dipping) method was utilized to achieve cost-efficiency and rapid production. The prototype deviates from traditional manufacturing by omitting the solder mask and silkscreen layers. Instead, reference designators (D1, D2, R1, R2) and orientation markings were etched directly into the copper pattern, resulting in a functional and integrated prototype SMD PCB.

f. Parts List & Invoices:

Item	Qty	Unit Price (TL)	Total (TL)	Status
Raspberry Pi 4 Model B	1	3314,05	3314,05	Purchased
ESP32 development board	1	248,60	248,60	Purchased but unused
Second Hand 3D Printer	1	6000	6000	Purchased
MS-B126 Vacummed SMD Holder	1	1203,15	1203,15	Planned
USB camera (top view)	1	749,25	749,25	Purchased
Power supply & wiring	1	192,40	192,40	Purchased
SD Card	1	419	419	Purchased
Raspberry Pi Camera	1	1093,63	1093,63	Purchased but unused
Arduino Set	1	629,90	629,90	Purchased
Custom PCB (control/IO)	1	540	540	Purchased



Ödeme Dekontu

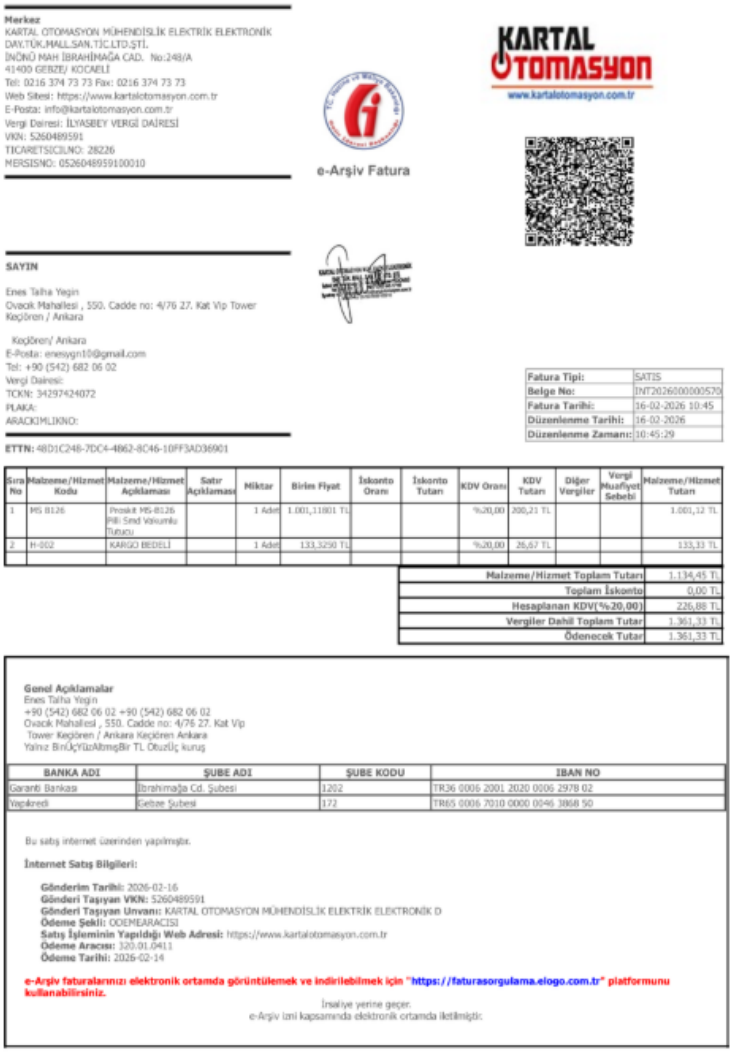
https://market.samm.com/ sitesinden yapmış olduğunuz **3.314,50 TL (Tek Çekim)** ödeme işleminiz tamamlandı.

İŞLEM ONAY TARİHİ	İŞLEM NUMARASI
14.01.2026 20:15	P176841092346333675918193181 25

Sipariş İçeriği:

Ikas Basket Amount (1)

Figure 4: Raspberry Pi Invoce



Merkez
KARTAL OTOMASYON MÜHENDİSLİK ELEKTRİK ELEKTRONİK DAY.TÜK.MALL.SAN.TİC.LTD.ŞTİ.
İNÖNÜ MAH İBRAHİMAĞA CAD. No:248/A
41400 GEBZE/ KOCAELİ
Tel: 0216 374 73 73 Fax: 0216 374 73 73
Web Sitesi: https://www.kartalotomasyon.com.tr
E-Posta: info@kartalotomasyon.com.tr
Vergi Dairesi: İLYASBEY VERGİ DAİRESİ
VKN: 5260489591
TICARETSICILNO: 28226
MERSISNO: 0526048959100010

e-Arşiv Fatura

KARTAL OTOMASYON
www.kartalotomasyon.com.tr

SAYIN

Enes Talha Yegin
Ovacık Mahallesi , 550. Cadde no: 4/76 27. Kat Vip Tower
Keçören / Ankara

Keçören/ Ankara
E-Posta: enesygn10@gmail.com
Tel: +90 (542) 682 06 02
Vergi Dairesi:
TCKN: 34297424072
PLAKA:
ARACKIMLIKNO:

Fatura Tipi:	SATIS
Belge No:	INT2026000000570
Fatura Tarihi:	16-02-2026 10:45
Düzenlenme Tarihi:	16-02-2026
Düzenlenme Zamanı:	10:45:29

ETTN: 48D1C248-7DC4-4862-8C46-10FF3AD36901

Sıra No	Malzeme/Hizmet Kodu	Malzeme/Hizmet Açıklaması	Satır Açıklaması	Miktar	Birim Fiyat	İskonto Oranı	İskonto Tutarı	KDV Oranı	KDV Tutarı	Diğer Vergiler	Vergi Muafiyet Sebebi	Malzeme/Hizmet Tutarı
1	MS 8126	Prosikit MS-8126 Pili Smd Yalıtımlı Tutucu		1 Adet	1.001,11801 TL			%20,00	200,21 TL			1.001,12 TL
2	H-002	KARGO BEDELİ		1 Adet	133,3250 TL			%20,00	26,67 TL			133,33 TL

Malzeme/Hizmet Toplam Tutarı	1.134,45 TL
Toplam İskonto	0,00 TL
Hesaplanan KDV(%20,00)	226,88 TL
Vergiler Dahil Toplam Tutar	1.361,33 TL
Ödenecek Tutar	1.361,33 TL

Genel Açıklamalar
Enes Talha Yegin
+90 (542) 682 06 02 +90 (542) 682 06 02
Ovacık Mahallesi , 550. Cadde no: 4/76 27. Kat Vip Tower Keçören / Ankara Keçören Ankara
Yalnız BinÜçYüzAltmışBir TL OtuzÜç kuruş

BANKA ADI	ŞUBE ADI	ŞUBE KODU	IBAN NO
Garanti Bankası	İbrahimağa Cd. Şubesi	1202	TR36 0006 2001 2020 0006 2978 02
Yapıkredi	Gebze Şubesi	172	TR65 0006 7010 0000 0046 3868 50

Bu satış internet üzerinden yapılmıştır.

İnternet Satış Bilgileri:

Gönderim Tarihi: 2026-02-16
Gönderi Taşıyan VKN: 5260489591
Gönderi Taşıyan Unvanı: KARTAL OTOMASYON MÜHENDİSLİK ELEKTRİK ELEKTRONİK D
Ödeme Şekli: ODEMEARACISI
Satış İşleminin Yapıldığı Web Adresi: https://www.kartalotomasyon.com.tr
Ödeme Aracısı: 320.01.0411
Ödeme Tarihi: 2026-02-14

e-Arşiv faturalarınızı elektronik ortamda görüntülemek ve indirilebilmek için "https://faturasorgulama.elogo.com.tr" platformunu kullanabilirsiniz.

İrsaliye yerine geçer.
e-Arşiv İzni kapsamında elektronik ortamda iletilmiştir.

Figure 5: Vacuum Invoice

Media Markt Ankara 2 Forum
MEDİA MARKT TURKEY TİCARET LİMİTED ŞİRKETİ
Media Markt Ankara 2 Forum
Ovacık Mah. Yozgat Bulvarı No: 97 İç Kapı No:137
Keçiören/Ankara
6130636884
Mersis No: 0613063688400256
Ticaret Sicil No: 616001

e-Arşiv Fatura

Enes
Yegin
TCKN/VKN: 11111111111

AŞIKPAŞA CAD.-
-keçiören/ANKARA
Türkiye

Tarih: 03. 02. 2026
Saat: 21:24:43

Fatura No: YV72026000003364
ETTN: 66082609-8d7b-446e-98b9-8f355544d740

Fatura Tipi: SATIS

Sistem No: 81559
Kasiyer: 16175 Çekmece: 3

Kasa No: 202
Mağaza: T006

588.1248181 KUCUK HAFIZA KARTLARI
SANDISK.SD ADAPTORLU SANDISK ULTRA 64GB MIC

% 20 419,00

KDV Toplam: 69,83
Toplam: 419,00

419,00

Diğer: KART GPRS ODEMESI

KDV Oranı % 20 KDV Tutarı 69,83

Net Mal Değeri: 349,17

YALNIZ Dörtyüzondokuz TL

9802190824505637 Club Kartı bu alışveriş için kullanıldı.

İrsaliye yerine geçer. E-Arşiv izni kapsamında elektronik olarak iletilmiştir.

Garanti Bankası Bakırköy Kurumsal Şube IBAN:TR18 0006 2000 3820 0006 2980 55

Media Markt Merkez Adresi : Yeşilce Mah. Eski Büyükdere Cad. No : 65 34418 Kağıthane / İstanbul

Figure 6: SD Cart Invoice

robotistan.com
"Türkiye'nin Maker Marketi"

Robotistan Elektronik Ticaret A.Ş.
MERKEZ:İKİTELLİ OSB MAH. ATATÜRK BLV.
HASEYAD 2 SANAYİ 108 6 BAŞAKŞEHİR/ İSTANBUL
Vergi Dairesi:İKİTELLİ VD.
Vergi Numarası: 735 090 71 05
Mail: info@robotistan.com **Web:** www.robotistan.com
Tel: 0850 766 0 425 - **Fax:** 0212 222 34 94
Ticaret Sicil No:774096**Mersis No:**0735090710500029

e-Arşiv Fatura

TS25011217897

Cari Ünvanı : Enes Yegin

Teslimat Adresi: Enes Yegin Ovacık
Mahallesi 550.Cadde no: 4/76 27.Kat Keçiören
/ Ankara Keçiören / Ankara

İlçe: Keçiören
İl: Ankara
E-Posta: enesygn10@gmail.com
Tel: +905426820602
VKN: 34297424072
Vergi Dairesi:
ETTN: 969AF768-0023-495C-A8C8-272495B0E4C5

Fatura Tipi: SATIS
Fatura Tarihi: 27-01-2026 09:19
Düzenlenme Tarihi: 27-01-2026
Düzenlenme Zamanı: 09:19:37
Belge No: RBT2026000008332

Ürün Kodu	Açıklama	KDV	Adet	B.Fiyat	Tutar
12958	Raspberry Pi Kamera Modül V2/ Menşei:CN	%20,00	1 Adet	794,704554 TL	794,70 TL
U000	Kargo Bedeli	%20,00	1 Adet	116,6583 TL	116,66 TL

Fatura Toplamı: 911,36 TL
İndirim Toplamı: 0,00 TL
Hesaplanan KDV(%20,00): 182,27 TL
Genel Toplam 1.093,63 TL

Genel Açıklamalar
Kredi Kartı
Garanti Bankası
Yalnız BinDoksanÜç Tl. Altmışüç kuruş
Bu satış internet üzerinden yapılmıştır.
e-Arşiv izni kapsamında oluşturulmuştur.
İrsaliye yerine geçer.
e-Arşiv izni kapsamında elektronik ortamda iletilmiştir.

İnternet Satış Bilgileri:
Gönderim Tarihi: 2026-01-26
Gönderi Taşıyan VKN: 9860008925
Gönderi Taşıyan Ünvanı: YURTİÇİ KARGO
Ödeme Şekli: KREDİKARTI/BANKAKARTI
Satış İşleminin Yapıldığı Web Adresi: www.robotistan.com
Ödeme Tarihi: 2026-01-27

Eksiksiz Teslim Eden
Adı - Soyadı

Eksiksiz Teslim Alan
Adı - Soyadı

- İşbu fatura muhtevıyatı mal ve faturanın ibrazından itibaren 8 gün içinde itiraz edilmediği takdirde aynen kabul edilmiş sayılır.
- Bu belgenin sevk edilen malla birlikte bulunması halinde ayrıca sevk irsaliyesi aranmaz. İrsaliye yerine geçer.
- Satış yapılan internet sitesi: www.robotistan.com

Banka Bilgileri

QNB Finansbank – İkitelli Organize Sanayi Şb.
Hesap Adı : Robotistan Elektronik Tic. A.ş. Hesap Türü : TL
IBAN : TR23 0011 1000 0000 0088 3633 11

QNB Finansbank – İkitelli Organize Sanayi Şb.
Hesap Adı : Robotistan Elektronik Tic. A.ş. Hesap Türü : USD
IBAN : TR32 0011 1000 0000 0088 6293 84

Figure 7: Raspberry Pi Camera Invoice

Demo Elektronik İsmail Calayır
Kale Mahallesi Konya Sokak 7/27-28
Altındağ/ Ankara
Tel: 8503042559
E-Posta: demopcbonline@gmail.com
Vergi Dairesi: kızılbey
TCKN: 26716402334
TICARETSICILNO:
MERSISNO:

e-Arşiv Fatura

Demo
0850 304 2559 | www.demoelektronik.com.tr

SAYIN
Enes Talha Yegin

Altındağ/ Ankara / Türkiye
E-Posta:
Tel: 05426820602
TCKN: 34297424072

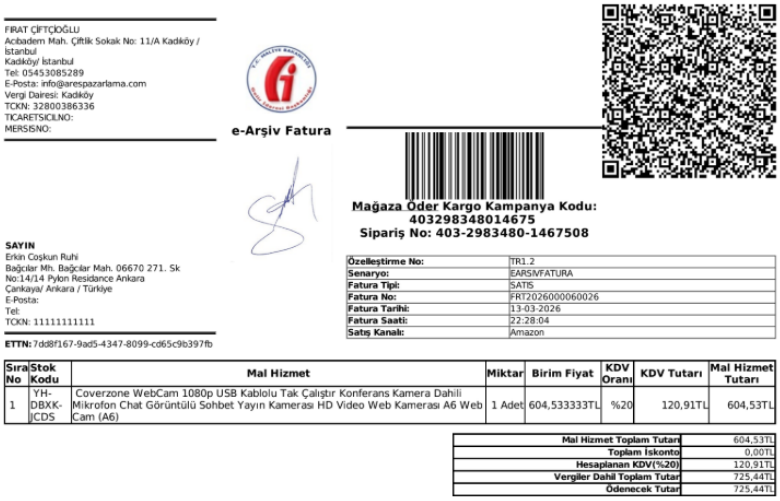
ETTN: 6f49ce36-7417-4644-88b8-f9e697968847

Özelleştirme No:	TR1.2
Senaryo:	EARSIVFATURA
Fatura Tipi:	SATIS
Fatura No:	EAR2026000000002
Fatura Tarihi:	21-02-2026
Fatura Saati:	17:05:31
Satış Kanalı:	Diğer

Sıra No	Stok Kodu	Stok Kodu (Alıcı)	Mal Hizmet	Miktar	Birim Fiyat	KDV Oranı	KDV Tutarı	Mal Hizmet Tutarı
1	1759738137140	***	PCB İmalat (3 farklı tasarım için 7şer adet)	1 Adet	450,00TL	%20	90,00TL	450,00TL

Mal Hizmet Toplam Tutarı	450,00TL
Toplam İskonto	0,00TL
Hesaplanan KDV(%20)	90,00TL
KDV Dahil Toplam Tutar	540,00TL
Ödenecek Tutar	540,00TL

Figure 8: PCB Invoice



FIRAT ÇİFTÇIOĞLU
Acıbadem Mah. Çiftlik Sokak No: 11/A Kadıköy / İstanbul
Kadıköy/ İstanbul
Tel: 05453085289
E-Posta: info@arespazarlama.com
Vergi Dairesi: Kadıköy
TCKN: 32800386336
TICARETSICILNO:
MERSISNO:

e-Arşiv Fatura

Mağaza Öder Kargo Kampanya Kodu:
403298348014675
Sipariş No: 403-2983480-1467508

SAYIN
Erkin Coşkun Ruhi
Bağcılar Mh. Bağcılar Mah. 06670 271. Sk
No:14/14 Pylon Residance Ankara
Çankaya/ Ankara / Türkiye
E-Posta:
Tel:
TCKN: 11111111111

ETTN: 7dd8f167-9ad5-4347-8099-cd65c9b397fb

Özelleştirme No:	TR1.2
Senaryo:	EARSIVFATURA
Fatura Tipi:	SATIS
Fatura No:	FRT2026000060026
Fatura Tarihi:	13-03-2026
Fatura Saati:	22:28:04
Satış Kanalı:	Amazon

Sıra No	Stok Kodu	Mal Hizmet	Miktar	Birim Fiyat	KDV Oranı	KDV Tutarı	Mal Hizmet Tutarı
1	YH-DBXK-ICDS	Coverzone WebCam 1080p USB Kablolu Tak Çalıştır Konferans Kamera Dahili Mikrofon Chat Görüntülü Sohbet Yayın Kameraası HD Video Web Kamerası A6 Web Cam (A6)	1 Adet	604,533333TL	%20	120,91TL	604,53TL

Mal Hizmet Toplam Tutarı	604,53TL
Toplam İskonto	0,00TL
Hesaplanan KDV(%20)	120,91TL
Vergiler Dahil Toplam Tutar	725,44TL
Ödenecek Tutar	725,44TL

Figure 9: Web Cam Invoice

Ödeme Dekontu

http://www.direnc.net/ sitesinden yapmış olduğunuz **248,02 TL (Tek Çekim)** ödeme işleminiz tamamlandı.

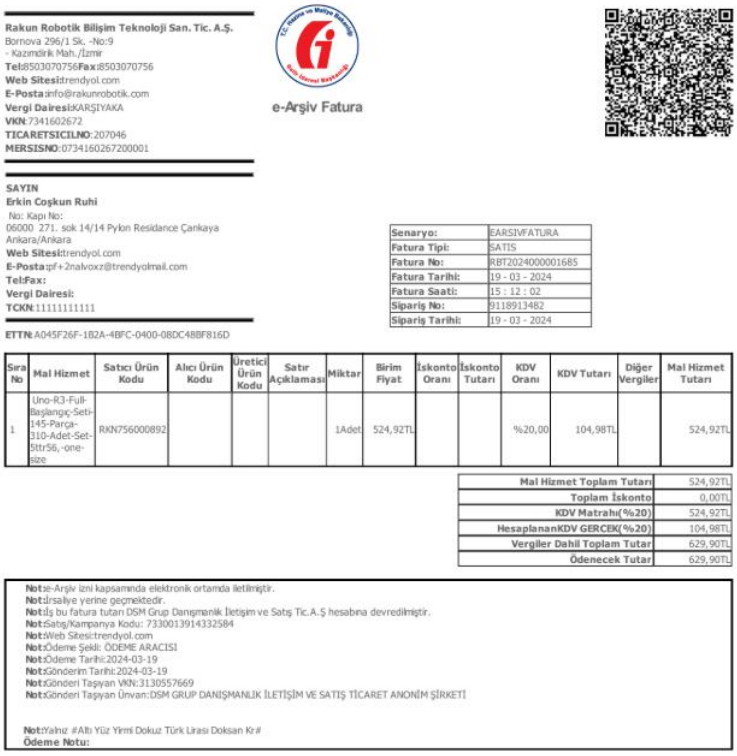
İŞLEM ONAY TARİHİ
14.01.2026 20:11

İŞLEM NUMARASI
P17684106773096326951764051228

Sipariş İçeriği:

ESP32-C3 Mini WiFi Bluetooth Board (1)
TS14011057626 (1)

Figure 10: ESP32 Invoice



Rakun Robotik Bilişim Teknoloji San. Tic. A.Ş.
Bornova 296/1 Sk. -No:9
- Kazımdirik Mah./İzmir
Tel:8503070756**Fax:**8503070756
Web Sitesi:trendyol.com
E-Posta:info@rakunrobotik.com
Vergi Dairesi:KARŞIYAKA
VKN:7341602672
TICARETSICILNO:207046
MERSISNO:0734160267200001

e-Arşiv Fatura

SAYIN
Erkin Coşkun Ruhi
No: Kapı No:
06000 271. sok 14/14 Pylon Residance Çankaya
Ankara/Ankara
Web Sitesi:trendyol.com
E-Posta:pf+2nalvoxz@trendyolmail.com
Tel:Fax:
Vergi Dairesi:
TCKN:11111111111

ETTN:A045F26F-1B2A-4BFC-0400-08DC48BF816D

Senaryo:	EARSIVFATURA
Fatura Tipi:	SATIS
Fatura No:	RBT2024000001685
Fatura Tarihi:	19 - 03 - 2024
Fatura Saati:	15 : 12 : 02
Sipariş No:	9118913482
Sipariş Tarihi:	19 - 03 - 2024

Sıra No	Mal Hizmet	Satıcı Ürün Kodu	Alıcı Ürün Kodu	Üretici Ürün Kodu	Satır Açıklaması	Miktar	Birim Fiyat	İskonto Oranı	İskonto Tutarı	KDV Oranı	KDV Tutarı	Diğer Vergiler	Mal Hizmet Tutarı
1	Uno-R3-Full-Başlangıç-Seti-145-Parça-310-Adet-Set-5ttr56,-one-size	RKN756000892				1Adet	524,92TL			%20,00	104,98TL		524,92TL

Mal Hizmet Toplam Tutarı	524,92TL
Toplam İskonto	0,00TL
KDV Matrahı(%20)	524,92TL
HesaplananKDV GERCEK(%20)	104,98TL
Vergiler Dahil Toplam Tutar	629,90TL
Ödenecek Tutar	629,90TL

Not:e-Arşiv izni kapsamında elektronik ortamda iletilmiştir.
Not:İrsaliye yerine geçmektedir.
Not:İş bu fatura tutarı DSM Grup Danışmanlık İletişim ve Satış Tic.A.Ş hesabına devredilmiştir.
Not: Satış/Kampanya Kodu: 7330013914332584
Not: Web Sitesi:trendyol.com
Not:Ödeme Şekli: ÖDEME ARACISI
Not:Ödeme Tarihi:2024-03-19
Not:Gönderim Tarihi:2024-03-19
Not:Gönderi Taşıyan VKN:3130557669
Not:Gönderi Taşıyan Ünvan:DSM GRUP DANIŞMANLIK İLETİŞİM VE SATIŞ TİCARET ANONİM ŞİRKETİ

Not: Yalnız #Altı Yüz Yirmi Dokuz Türk Lirası Doksan Kr.#
Ödeme Notu:

Figure 11: Arduino Set Invoice

ROBOCOMBO TEKNOLOJİ ÜRÜNLERİ SANAYİ VE TİCARET LİMİTED ŞİRKETİ
ORUÇREİS MAH. TEKSTİLKENT CAD. TEKSTİLKENT A19 BLOK NO: 15
34235 ESENLER/ İSTANBUL
Tel: 0552 550 76 26 Fax:
Web Sitesi: www.robocombo.com
E-Posta: info@robocombo.com
Vergi Dairesi: ATIŞALANI
VKN: 7350886518

e-Arşiv Fatura

SAYIN
GAYE ERDOĞAN
1238/2 SOK. NO 7 ZİRVELİFE SİTESİ A BLOK 21 NUMARA

YENİMAHALLE YENİMAHALLE/ ANKARA TÜRKİYE
E-Posta: ge37242@gmail.com
Tel: 905551405075 Fax:
Vergi Dairesi:
TCKN: 11111111111

Özelleştirme No	TR1.2
Senaryo	EARSIVFATURA
Fatura Tipi	SATIS
Fatura No	EAR2026000002499
Fatura Tarihi	23 - 02 - 2026 00:00
Sipariş No	514815
Sipariş Tarihi	21 - 02 - 2026

ETTN: 010E8F52-B70E-4469-B638-802131CFEC1B

Sıra No	Mal Hizmet	Miktar	Birim Fiyat	KDV Oranı	KDV Tutarı	Mal Hizmet Tutarı
1	Raspberry Pi için FFC 60cm Kablo	1,0Adet	44TL	%20,00	8,80TL	44,00TL
2	Kargo Ücreti	1,0Adet	116,66670227TL	%20,00	23,33TL	116,67TL

Mal Hizmet Toplam Tutarı	160,67TL
Toplam İskonto	0,00TL
Hesaplanan Kdv(%20)	32,13TL
Vergiler Dahil Toplam Tutar	192,80TL
Ödenecek Tutar	192,80TL

YALNIZ: YÜZ DOKSAN İKİ TL SEKSEN KRŞ.
Not: YALNIZ : YÜZDOKSANİKİ TL SEKSEN Kr.

Banka Adı	Şube	Hesap No	IBAN
KUVEYT TÜRK	GÜNEŞLİ-ÇARŞI	93857465	TR62 0020 5000 0938 5746 5000 03

İrsaliye yerine geçer

e-Arşiv izni kapsamında elektronik ortamda iletilmiştir.
Bu satış internet üzerinden yapılmıştır

Web Adresi	www.robocombo.com	**Ödeme Şekli**	KREDIKARTI/BANKAKARTI	**Ödeme Tarihi**	2026-02-23
Gönderiyi Taşıyan	HEPSİJET KARGO	**Taşıyıcı VKN/TCKN**	2650701090	**Gönderim Tarihi**	2026-02-23

Figure 12: Cable Invoice

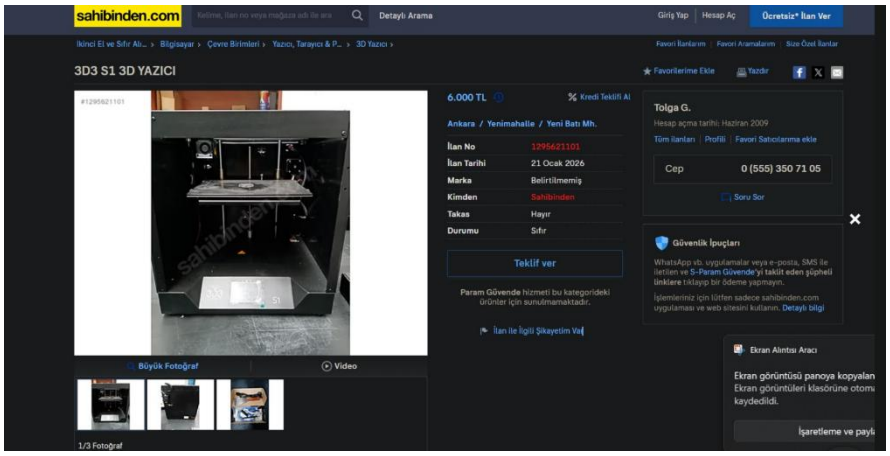


Figure 13: 3D Printer Invoice

k. Accomplishment Checklist**:**

- Fabrication of Surface Mount PCB: Successful
- Autonomous Picking and Precise Placement of Components: Successful
- Functional and Accurate User Interface (UI) Operation: Successful
- Reliable and Continuous System-Wide Operation: Successful
- Open Source Documentation and GitHub Repository Sharing: Successful
- [Bonus] Determination and Real-Time Display of Placement Accuracy: Successful
- [Bonus] Electrical Testing for 4 Additional Resistors: Successful

Realisation Gains and Market Outcomes

Realizing this autonomous pick-and-place project has provided a comprehensive understanding of Mechatronic System Integration. We gained significant experience in managing high-level tasks on a Raspberry Pi while maintaining low-level hardware control via Arduino/Atmega platforms. The necessity of transitioning from an ESP32 to an Arduino Nano and from a CSI camera to a USB interface taught me the importance of modular design and system adaptability in the face of hardware failures. Developing the Fixed-Coordinate IoU and Area-Ratio algorithms allowed me to bridge the gap between theoretical image processing and practical, real-time quality control (AOI).

9.1 Market Outcomes and Potential

A similar project in the professional market would serve as a Low-Cost Desktop SMT Assembly Solution for startups and R&D laboratories. While industrial SMT lines cost hundreds of thousands of dollars, a modular, Raspberry Pi-based system offers an accessible entry point for small-scale PCBA prototyping. The integrated AOI (Automatic Optical Inspection) feature ensures that manual errors are eliminated early in the production cycle, significantly reducing waste and rework costs in electronics manufacturing.

9.2 Requirements for Market Success

To transition this prototype into a successful commercial product, the following enhancements would be required:

- **Enhanced Speed and Precision:** Replacing standard stepper motors with high-resolution **closed-loop servo motors** and linear encoders to achieve sub-micron placement precision.
- **AI-Driven Component Recognition:** Utilizing Deep Learning (CNNs) instead of traditional template matching to allow the system to recognize a wider variety of components (ICs, QFPs, BGA) without manual calibration.
- **Smart Feeder Systems:** Developing an automated tape-and-reel feeder system to allow continuous operation without manual component loading.
- **Industrial Connectivity:** Integrating OPC-UA or MQTT protocols to make the device compatible with Industry 4.0 standards and smart factory ecosystems

References

1. • **Bradski, G., & Kaehler, A. (2008).** *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.
2. • **Rosebrock, A. (2016).** "Intersection over Union (IoU) for object detection." *PyImageSearch*. [Online]. Available: https://pyimagesearch.com
3. • **Everingham, M., et al. (2010).** "The Pascal Visual Object Classes (VOC) Challenge." *International Journal of Computer Vision*.
4. • **IPC International (2017).** *IPC-A-610G: Acceptability of Electronic Assemblies*.
5. • **Taiwan Semiconductor (2022).** *S3A SERIES: Surface Mount Rectifier Datasheet* Mouser Electronics.
6. • **DSK (DSK Semiconductor) (2015).** *SK510C: Surface Mount Schottky Barrier Rectifier Datasheet*.
7. • **Pro'sKit MS-B126 SMD Vacuum Pick up Tool** https://www.prokits.com.tw/Product/MS-B126/
8. • **Arduino Uno Rev3** Arduino. *Arduino Uno Rev3 Datasheet*. https://store.arduino.cc/products/arduino-uno-rev3
9. • **Raspberry Pi 4 Model B** Raspberry Pi Foundation. *Raspberry Pi 4 Model B Datasheet*. https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008341-DS-1-raspberry-pi-4-datasheet.pdf?disposition=inline

10. • **Arduino Nano**
Arduino. *Arduino Nano Datasheet*.
https://store.arduino.cc/products/arduino-nano

Appendix

a. Codes:

placement_verifier.py

```
#!/usr/bin/env python3

import argparse

import cv2

import json

import math

import os

import time

import numpy as np

import subprocess


BASE_DIR = "/home/pi/vision"


EMPTY_IMG = os.path.join(BASE_DIR, "empty_pcb.jpg")

TEST_IMG  = os.path.join(BASE_DIR, "test_pcb.jpg")


OUT_JSON  = os.path.join(BASE_DIR, "placement_result.json")

OUT_DEBUG = os.path.join(BASE_DIR, "placement_debug.jpg")

OUT_ALIGN = os.path.join(BASE_DIR, "debug_aligned.jpg")


WEBCAM_INDEX    = 0

WEBCAM_WARMUP_S = 2.0
```

```
DIFF_THRESH = 20
MIN_BLOB    = 25
TM_METHOD   = cv2.TM_CCOEFF_NORMED

SHIFT_PENALTY = 1.2
ROT_PENALTY   = 0.5
MAX_SHIFT     = 18
MAX_ROT       = 20
MIN_SCORE_OK  = 65

GREEN_THRESH  = 75
YELLOW_THRESH = 55

ROI_W = 72
ROI_H = 84

def log(m: str):
    print("[AOI]", m)

def get_perfect_img(board_mode: str) -> str:
    if board_mode == "2d2r":
        return os.path.join(BASE_DIR, "perfect_2d2r_pcb.jpg")
    elif board_mode == "4r":
        return os.path.join(BASE_DIR, "perfect_4r_pcb.jpg")
    raise RuntimeError(f"Bilinmeyen board_mode: {board_mode}")

def apply_camera_settings():
```

```
    cmds = [
        ["v4l2-ctl", "-d", "/dev/video0", "-c", "auto_exposure=1"],
        ["v4l2-ctl", "-d", "/dev/video0", "-c", "exposure_time_absolute=1200"],
        ["v4l2-ctl", "-d", "/dev/video0", "-c", "brightness=0"],
        ["v4l2-ctl", "-d", "/dev/video0", "-c", "contrast=34"],
        ["v4l2-ctl", "-d", "/dev/video0", "-c", "gain=0"],
        ["v4l2-ctl", "-d", "/dev/video0", "-c", "white_balance_automatic=0"],
        ["v4l2-ctl", "-d", "/dev/video0", "-c", "white_balance_temperature=4000"],
    ]

    for cmd in cmds:
        try:
            subprocess.run(cmd, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
        except Exception as e:
            print(f"[AOI] Kamera ayarı uygulanamadı: {e}")


def get_calib_json(board_mode: str) -> str:
    if board_mode == "2d2r":
        return os.path.join(BASE_DIR, "placement_calibration_2d2r.json")
    elif board_mode == "4r":
        return os.path.join(BASE_DIR, "placement_calibration_4r.json")
    raise RuntimeError(f"Bilinmeyen board_mode: {board_mode}")


def roi_from_center(cx: int, cy: int, w: int = ROI_W, h: int = ROI_H):
    x = int(cx - w // 2)
    y = int(cy - h // 2)
```

```
    return {"x": x, "y": y, "w": w, "h": h}


def get_default_components(board_mode: str):
    if board_mode == "2d2r":
        centers = [
            {"name": "D1", "cx": 520, "cy": 96},
            {"name": "D2", "cx": 640, "cy": 96},
            {"name": "R1", "cx": 520, "cy": 278},
            {"name": "R2", "cx": 640, "cy": 278},
        ]
    elif board_mode == "4r":
        centers = [
            {"name": "D1", "cx": 543, "cy": 106},
            {"name": "D2", "cx": 667, "cy": 106},
            {"name": "R1", "cx": 543, "cy": 278},
            {"name": "R2", "cx": 667, "cy": 278},
        ]
    else:
        raise RuntimeError(f"Bilinmeyen board_mode: {board_mode}")


    out = []
    for c in centers:
        out.append({
            "name": c["name"],
            "center": {"cx": c["cx"], "cy": c["cy"]},
            "roi": roi_from_center(c["cx"], c["cy"])
        })
```

```
    return out


def refresh_component_rois(components):
    refreshed = []
    for comp in components:
        if "center" in comp:
            cx = comp["center"]["cx"]
            cy = comp["center"]["cy"]
            refreshed.append({
                "name": comp["name"],
                "center": {"cx": cx, "cy": cy},
                "roi": roi_from_center(cx, cy)
            })
        else:
            refreshed.append(comp)
    return refreshed


def get_status_and_color(score: float, present: bool):
    if not present:
        return "missing", (0, 0, 255)

    if score >= GREEN_THRESH:
        return "good", (0, 255, 0)
    elif score >= YELLOW_THRESH:
        return "warning", (0, 255, 255)
    else:
        return "bad", (0, 0, 255)
```

```
def read_img(p: str) -> np.ndarray:
    im = cv2.imread(p)
    if im is None:
        raise RuntimeError("Görüntü okunamadı: " + p)
    return im

def load_json(p: str) -> dict:
    with open(p, encoding="utf-8") as f:
        return json.load(f)

def save_json(p: str, d: dict):
    with open(p, "w", encoding="utf-8") as f:
        json.dump(d, f, indent=2, ensure_ascii=False)

def order_pts(pts) -> np.ndarray:
    pts = np.array(pts, dtype=np.float32)
    s = pts.sum(axis=1)
    diff = np.diff(pts, axis=1)

    tl = pts[np.argmin(s)]
    br = pts[np.argmax(s)]
    tr = pts[np.argmin(diff)]
    bl = pts[np.argmax(diff)]

    return np.array([tl, tr, br, bl], dtype=np.float32)
```

```
def warp(img: np.ndarray, corners, w: int, h: int) -> np.ndarray:
    src = order_pts(corners)
    dst = np.array([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]], dtype=np.float32)
    M = cv2.getPerspectiveTransform(src, dst)
    return cv2.warpPerspective(img, M, (w, h))

def align_global(ref: np.ndarray, img: np.ndarray) -> np.ndarray:
    g1 = cv2.cvtColor(ref, cv2.COLOR_BGR2GRAY)
    g2 = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    orb = cv2.ORB_create(5000)
    kp1, des1 = orb.detectAndCompute(g1, None)
    kp2, des2 = orb.detectAndCompute(g2, None)

    if des1 is None or des2 is None:
        log("align_global: descriptor yok, hizalama atlandı")
        return img

    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
    matches = bf.match(des1, des2)
    matches = sorted(matches, key=lambda x: x.distance)[:200]

    if len(matches) < 10:
        log("align_global: yeterli eşleşme yok, hizalama atlandı")
        return img

    pts1 = np.float32([kp1[m.queryIdx].pt for m in matches])
```

```
    pts2 = np.float32([kp2[m.trainIdx].pt for m in matches])

    H, mask = cv2.findHomography(pts2, pts1, cv2.RANSAC, 5)
    if H is None:
        log("align_global: homografi hesaplanamadı")
        return img

    inliers = int(mask.sum()) if mask is not None else 0
    log(f"align_global: {inliers} inlier ile hizalandı")
    return cv2.warpPerspective(img, H, (ref.shape[1], ref.shape[0]))

def diff_mask(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    g1 = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    g2 = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY)

    d = cv2.absdiff(g1, g2)
    _, th = cv2.threshold(d, DIFF_THRESH, 255, cv2.THRESH_BINARY)

    k = np.ones((3, 3), np.uint8)
    th = cv2.morphologyEx(th, cv2.MORPH_OPEN, k)
    th = cv2.morphologyEx(th, cv2.MORPH_CLOSE, k)

    num, labels, stats, _ = cv2.connectedComponentsWithStats(th)
    out = np.zeros_like(th)

    for i in range(1, num):
        if stats[i, cv2.CC_STAT_AREA] >= MIN_BLOB:
```

```
        out[labels == i] = 255

    return out

def centroid(mask: np.ndarray):
    m = cv2.moments(mask)
    if m["m00"] == 0:
        return None
    return (int(m["m10"] / m["m00"]), int(m["m01"] / m["m00"]))

def rotation_angle(mask: np.ndarray):
    pts = np.column_stack(np.where(mask > 0))
    if len(pts) < 10:
        return None

    pts = pts.astype(np.float32)
    pts -= np.mean(pts, axis=0)

    cov = np.cov(pts, rowvar=False)
    eigvals, eigvecs = np.linalg.eig(cov)
    major = eigvecs[:, np.argmax(eigvals)]

    return float(math.degrees(math.atan2(major[0], major[1])))

def template_from_mask(mask: np.ndarray):
    ys, xs = np.where(mask > 0)
    if len(xs) == 0:
```

```
        return None, None

    x1, x2 = int(xs.min()), int(xs.max())
    y1, y2 = int(ys.min()), int(ys.max())

    return mask[y1:y2 + 1, x1:x2 + 1], (x1, y1, x2 - x1 + 1, y2 - y1 + 1)

def iou_score(a: np.ndarray, b: np.ndarray) -> float:
    a = a > 0
    b = b > 0

    inter = np.logical_and(a, b).sum()
    union = np.logical_or(a, b).sum()

    if union == 0:
        return 0.0

    return float(100.0 * inter / union)

def match_template_score(roi_gray: np.ndarray, tmpl: np.ndarray):
    if tmpl is None or roi_gray is None:
        return 0.0, (0, 0)

    rh, rw = roi_gray.shape[:2]
    th, tw = tmpl.shape[:2]

    if th > rh or tw > rw:
```

```
        return 0.0, (0, 0)

    res = cv2.matchTemplate(roi_gray, tmpl, TM_METHOD)
    _, maxv, _, maxloc = cv2.minMaxLoc(res)
    return float(maxv), maxloc

def ncc_similarity(a: np.ndarray, b: np.ndarray) -> float:
    ga = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY).astype(np.float32)
    gb = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY).astype(np.float32)
    result = cv2.matchTemplate(ga, gb, cv2.TM_CCOEFF_NORMED)
    return float(result[0, 0])

def capture_test_image(out_path: str = TEST_IMG):
    log(f"Webcam'den görüntü alınıyor (index={WEBCAM_INDEX})...")

    apply_camera_settings()
    time.sleep(0.3)

    cap = cv2.VideoCapture(WEBCAM_INDEX)
    if not cap.isOpened():
        raise RuntimeError(f"Webcam açılamadı (index={WEBCAM_INDEX})")

    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

    t0 = time.time()
    while time.time() - t0 < WEBCAM_WARMUP_S:
```

```
        cap.read()

    for _ in range(8):
        cap.read()
        time.sleep(0.03)

    ret, frame = cap.read()
    cap.release()

    if not ret or frame is None:
        raise RuntimeError("Webcam'den kare alınamadı")

    cv2.imwrite(out_path, frame)
    log(f"Test görüntüsü kaydedildi: {out_path}")
    return frame

def run_calibrate(board_mode: str):
    calib_json = get_calib_json(board_mode)
    log(f"Kalibrasyon modu: board_mode={board_mode}")

    data = {
        "board_mode": board_mode,
        "board_corners": [[0, 0], [1280, 0], [1280, 720], [0, 720]],
        "warp_size": {"width": 1280, "height": 720},
        "roi_size": {"w": ROI_W, "h": ROI_H},
        "components": get_default_components(board_mode)
    }
```

```
    save_json(calib_json, data)
    log(f"Kalibrasyon kaydedildi: {calib_json}")


def run_verify(board_mode: str, auto_capture: bool):
    calib_json = get_calib_json(board_mode)
    log(f"Doğrulama modu: board_mode={board_mode},
auto_capture={auto_capture}")


    if not os.path.exists(calib_json):
        raise RuntimeError(
            f"Kalibrasyon dosyası yok: {calib_json}\n"
            f"Önce kalibrasyon çalıştır:\n"
            f"  python {__file__} --mode calibrate --board-mode {board_mode}"
        )


    calib = load_json(calib_json)
    calib["components"] = refresh_component_rois(calib["components"])


    w = calib["warp_size"]["width"]
    h = calib["warp_size"]["height"]


    if auto_capture:
        capture_test_image(TEST_IMG)


    perfect_img_path = get_perfect_img(board_mode)
```

```
empty   = read_img(EMPTY_IMG)
perfect = read_img(perfect_img_path)
test    = read_img(TEST_IMG)

empty   = warp(empty, calib["board_corners"], w, h)
perfect = warp(perfect, calib["board_corners"], w, h)
test    = warp(test, calib["board_corners"], w, h)

test = align_global(perfect, test)
cv2.imwrite(OUT_ALIGN, test)

debug = test.copy()
slots = {}
scores = []

for comp in calib["components"]:
    name = comp["name"]
    r = comp["roi"]
    x, y, wc, hc = r["x"], r["y"], r["w"], r["h"]

    e_roi = empty[y:y + hc, x:x + wc]
    p_roi = perfect[y:y + hc, x:x + wc]
    t_roi = test[y:y + hc, x:x + wc]

    if e_roi.size == 0 or p_roi.size == 0 or t_roi.size == 0:
        log(f"HATA: {name} ROI görüntü dışına taşıyor")
        status, color = get_status_and_color(0.0, False)
```

```
    slots[name] = {
        "present": False,
        "score": 0.0,
        "status": status,
        "pos_error_px": None,
        "angle_error_deg": None,
        "iou": 0.0,
        "template_score": 0.0,
        "shift_x": None,
        "shift_y": None,
        "ok": False,
    }
    scores.append(0.0)
    cv2.rectangle(debug, (x, y), (x + wc, y + hc), color, 2)
    cv2.putText(debug, f"{name}:ROI_ERR", (x, y - 8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)
    continue

ideal_mask = diff_mask(p_roi, e_roi)
ideal_area = int((ideal_mask > 0).sum())

if ideal_area == 0:
    log(f"UYARI: {name} için ideal maske boş")
    status, color = get_status_and_color(0.0, False)
    slots[name] = {
        "present": False,
        "score": 0.0,
```

```
        "status": status,
        "pos_error_px": None,
        "angle_error_deg": None,
        "iou": 0.0,
        "template_score": 0.0,
        "shift_x": None,
        "shift_y": None,
        "ok": False,
    }
    scores.append(0.0)
    cv2.rectangle(debug, (x, y), (x + wc, y + hc), color, 2)
    cv2.putText(debug, f"{name}:NO_IDEAL", (x, y - 8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)
    continue

detected = diff_mask(t_roi, e_roi)
detected_area = int((detected > 0).sum())

sim_to_perfect = ncc_similarity(t_roi, p_roi)
sim_to_empty   = ncc_similarity(t_roi, e_roi)

min_present_area = max(20, int(0.20 * ideal_area))
is_present = (
    sim_to_perfect > sim_to_empty
    and detected_area >= min_present_area
)
```

```
if detected_area < min_present_area:
    is_present = False

log(
    f"{name}: sim_to_perfect={sim_to_perfect:.3f}  "
    f"sim_to_empty={sim_to_empty:.3f}  "
    f"ideal_area={ideal_area} detected_area={detected_area} "
    f"min_present_area={min_present_area} present={is_present}"
)

if not is_present:
    log(f"UYARI: {name} tespit edilemedi (eksik komponent)")
    status, color = get_status_and_color(0.0, False)
    slots[name] = {
        "present": False,
        "score": 0.0,
        "status": status,
        "pos_error_px": None,
        "angle_error_deg": None,
        "iou": 0.0,
        "template_score": 0.0,
        "shift_x": None,
        "shift_y": None,
        "ok": False,
    }
    scores.append(0.0)
    cv2.rectangle(debug, (x, y), (x + wc, y + hc), color, 2)
```

```
    cv2.putText(debug, f"{name}:MISSING", (x, y - 8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)
    continue

tmpl, _ = template_from_mask(ideal_mask)
t_gray = cv2.cvtColor(t_roi, cv2.COLOR_BGR2GRAY)
score_tm, _ = match_template_score(t_gray, tmpl)

iou = iou_score(ideal_mask, detected)

c1 = centroid(ideal_mask)
c2 = centroid(detected)

shift = sx = sy = None
if c1 and c2:
    sx = c2[0] - c1[0]
    sy = c2[1] - c1[1]
    shift = float(math.sqrt(sx * sx + sy * sy))

r1 = rotation_angle(ideal_mask)
r2 = rotation_angle(detected)
rot_err = float(abs(r1 - r2)) if (r1 is not None and r2 is not None) else None

final = iou
if shift is not None:
    final -= shift * SHIFT_PENALTY
if rot_err is not None:
```

```
    final -= rot_err * ROT_PENALTY

final = float(max(0.0, min(100.0, final)))

ok = (
    final >= MIN_SCORE_OK
    and (shift is None or shift <= MAX_SHIFT)
    and (rot_err is None or rot_err <= MAX_ROT)
)

status, color = get_status_and_color(final, True)

cv2.rectangle(debug, (x, y), (x + wc, y + hc), color, 2)
cv2.putText(debug, f"{name}:{final:.1f}", (x, y - 8),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2)

if c1:
    cv2.circle(debug, (x + c1[0], y + c1[1]), 4, (0, 255, 0), -1)
if c2:
    cv2.circle(debug, (x + c2[0], y + c2[1]), 4, (0, 0, 255), -1)
if c1 and c2:
    cv2.line(debug, (x + c1[0], y + c1[1]), (x + c2[0], y + c2[1]), (255, 255, 0), 2)

slots[name] = {
    "present": True,
    "score": round(final, 2),
    "status": status,
```

```
        "pos_error_px": round(shift, 2) if shift is not None else None,
        "angle_error_deg": round(rot_err, 2) if rot_err is not None else None,
        "iou": round(iou, 2),
        "template_score": round(score_tm, 3),
        "shift_x": int(sx) if sx is not None else None,
        "shift_y": int(sy) if sy is not None else None,
        "ok": bool(ok),
    }
    scores.append(final)

overall_accuracy = float(round(sum(scores) / len(scores), 2)) if scores else 0.0

cv2.putText(
    debug,
    f"OVERALL:{overall_accuracy:.1f}%",
    (20, 40),
    cv2.FONT_HERSHEY_SIMPLEX,
    1.0,
    (255, 255, 255),
    3
)
cv2.imwrite(OUT_DEBUG, debug)

out = {
    "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
    "board_mode": board_mode,
    "overall_accuracy": overall_accuracy,
```

```
        "slots": slots,
    }
    save_json(OUT_JSON, out)

    log(f"Doğrulama tamamlandı. Overall: %{overall_accuracy}")
    log(f"Sonuç: {OUT_JSON}")
    log(f"Debug: {OUT_DEBUG}")

    return out

def parse_args():
    parser = argparse.ArgumentParser(description="PCB Yerleştirme Doğrulama")
    parser.add_argument(
        "--mode",
        choices=["calibrate", "verify"],
        default="verify",
        help="calibrate: kalibrasyon JSON'unu güncelle | verify: yerleştirme kontrolü
yap",
    )
    parser.add_argument(
        "--board-mode",
        default="2d2r",
        choices=["2d2r", "4r"],
        help="2d2r: 2 diyot + 2 direnç | 4r: 4 direnç",
    )
    parser.add_argument(
        "--auto-capture",
```

```
        action="store_true",
        help="Doğrulama öncesi webcam'den otomatik görüntü al",
    )
    return parser.parse_args()

def main():
    args = parse_args()

    if args.mode == "calibrate":
        run_calibrate(args.board_mode)

    elif args.mode == "verify":
        result = run_verify(
            board_mode=args.board_mode,
            auto_capture=args.auto_capture,
        )

        print("\n========== SONUÇ ==========")
        print(f"Overall Accuracy : %{result['overall_accuracy']}")

        slot_order = ["D1", "D2", "R1", "R2"]

        for slot_name in slot_order:
            info = result["slots"].get(slot_name)
            if info:
                print(
                    f"  {slot_name}: present={info['present']} | "
```

```
                f"status={info['status']} | "
                f"score=%{info['score']} | "
                f"pos_error_px={info['pos_error_px']} | "
                f"angle_error_deg={info['angle_error_deg']}"
            )

    if __name__ == "__main__":
        main()
```

capture_samples.py

```
#!/usr/bin/env python3
import cv2
import time
import os
import subprocess

SAVE_DIR = "/home/pi/vision"
CAM_INDEX = 0
VIDEO_DEV = "/dev/video0"

def apply_camera_settings():
    cmds = [
        ["v4l2-ctl", "-d", VIDEO_DEV, "-c", "auto_exposure=1"],
        ["v4l2-ctl", "-d", VIDEO_DEV, "-c", "exposure_time_absolute=1200"],
        ["v4l2-ctl", "-d", VIDEO_DEV, "-c", "brightness=0"],
        ["v4l2-ctl", "-d", VIDEO_DEV, "-c", "contrast=34"],
        ["v4l2-ctl", "-d", VIDEO_DEV, "-c", "gain=0"],
        ["v4l2-ctl", "-d", VIDEO_DEV, "-c", "white_balance_automatic=0"],
```

```
        ["v4l2-ctl", "-d", VIDEO_DEV, "-c", "white_balance_temperature=4000"],
    ]

    for cmd in cmds:
        try:
            subprocess.run(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
        except Exception as e:
            print(f"Kamera ayarı uygulanamadı: {e}")

def capture_pcb(file_name):
    os.makedirs(SAVE_DIR, exist_ok=True)

    apply_camera_settings()
    time.sleep(0.3)

    cap = cv2.VideoCapture(CAM_INDEX)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

    if not cap.isOpened():
        print("Hata: Kamera açılamadı!")
        return

    time.sleep(2.0)  # Kameranın sabitlenmesi için bekleme

    # Birkaç kare flush
    for _ in range(10):
```

```
        cap.read()
        time.sleep(0.03)

    ret, frame = cap.read()

    if ret and frame is not None:
        file_path = os.path.join(SAVE_DIR, f"{file_name}.jpg")
        cv2.imwrite(file_path, frame)
        print(f"Başarıyla kaydedildi: {file_path}")
    else:
        print("Hata: Kamera görüntüsü alınamadı!")

    cap.release()

step = input(
    "Hangi resmi çekeceksin? "
    "(empty / perfect_2d2r / perfect_4r / test): "
).strip().lower()

file_map = {
    "empty": "empty_pcb",
    "perfect_2d2r": "perfect_2d2r_pcb",
    "perfect_4r": "perfect_4r_pcb",
    "test": "test_pcb",
}

if step in file_map:
```

```
    capture_pcb(file_map[step])
else:
    print("Geçersiz seçenek! Lütfen empty, perfect_2d2r, perfect_4r veya test yazın.")
```

main_system.py

```
#!/usr/bin/env python3
import re
import time
import serial
import os
import json
import subprocess

# ============================================================
# WEB LOG CALLBACK
# ============================================================
LOG_FULL_CB = None
LOG_USER_CB = None

def set_log_callbacks(full_cb=None, user_cb=None):
    global LOG_FULL_CB, LOG_USER_CB
    LOG_FULL_CB = full_cb
    LOG_USER_CB = user_cb

def log_full(msg: str):
    print(msg)
    if LOG_FULL_CB:
```

```
        try:
            LOG_FULL_CB(msg)
        except Exception:
            pass

def log_user(msg: str):
    print(msg)
    if LOG_USER_CB:
        try:
            LOG_USER_CB(msg)
        except Exception:
            pass

# ============================================================
# PORTLAR
# ============================================================
PRINTER_PORT = "/dev/ttyUSB1"
ARDUINO_PORT = "/dev/ttyACM0"
NANO_PORT    = "/dev/ttyUSB0"

PRINTER_BAUD = 115200
ARDUINO_BAUD = 9600
NANO_BAUD    = 115200

# ============================================================
# YERLEŞTİRME DOĞRULAMA
# ============================================================
```

```
PLACEMENT_VERIFIER_PY = "/home/pi/vision/placement_verifier.py"
PLACEMENT_VERIFIER_PYTHON = "/home/pi/vision/.venv/bin/python"
PLACEMENT_RESULT_JSON = "/home/pi/vision/placement_result.json"
PLACEMENT_DEBUG_IMG = "/home/pi/vision/placement_debug.jpg"

def get_placement_calib_json(mode: int) -> str:
    if mode == 1:
        return "/home/pi/vision/placement_calibration_2d2r.json"
    elif mode == 2:
        return "/home/pi/vision/placement_calibration_4r.json"
    raise RuntimeError(f"Bilinmeyen mod: {mode}")

# ============================================================
# FARK
# ============================================================
X_Diff = 0
Y_Diff = 0

# ============================================================
# REFERANS / HOME / SAFE
# ============================================================
HOME_X = 5.0
HOME_Y = 5.0
SAFE_Z = 25.0

# ============================================================
# DİYOT ALMA YERLERİ
```

```
# ==========================================================
DIODE_PICK1_X = 46.8 + X_Diff
DIODE_PICK1_Y = 79.5 + Y_Diff
DIODE_PICK1_Z = 7.0

DIODE_PICK2_X = 65.0 + X_Diff
DIODE_PICK2_Y = 79.5 + Y_Diff
DIODE_PICK2_Z = 7.0

DIODE_PICK_LIST = [
    ("DIODE_PICK1", DIODE_PICK1_X, DIODE_PICK1_Y, DIODE_PICK1_Z),
    ("DIODE_PICK2", DIODE_PICK2_X, DIODE_PICK2_Y, DIODE_PICK2_Z),
]

# ==========================================================
# DİRENÇ ALMA YERLERİ
# ==========================================================
RES_PICK1_X = 102.9 + X_Diff
RES_PICK1_Y = 90.0 + Y_Diff
RES_PICK1_Z = 5.0

RES_PICK2_X = 102.9 + X_Diff
RES_PICK2_Y = 86.0 + Y_Diff
RES_PICK2_Z = 5.0

RES_PICK3_X = 102.9 + X_Diff
RES_PICK3_Y = 82.0 + Y_Diff
```

```
RES_PICK3_Z = 5.0

RES_PICK4_X = 103.0 + X_Diff
RES_PICK4_Y = 78.0 + Y_Diff
RES_PICK4_Z = 5.0

RESISTOR_PICK_LIST = [
    ("R_PICK1", RES_PICK1_X, RES_PICK1_Y, RES_PICK1_Z),
    ("R_PICK2", RES_PICK2_X, RES_PICK2_Y, RES_PICK2_Z),
    ("R_PICK3", RES_PICK3_X, RES_PICK3_Y, RES_PICK3_Z),
    ("R_PICK4", RES_PICK4_X, RES_PICK4_Y, RES_PICK4_Z),
]

# ============================================================
# TEST İSTASYONLARI
# ============================================================
DIODE_TEST_X = 30.20 + X_Diff
DIODE_TEST_Y = 113.5 + Y_Diff
DIODE_TEST_Z = 10.0

RES_TEST_X = 20.90 + X_Diff
RES_TEST_Y = 181.0 + Y_Diff
RES_TEST_Z = 9.50

# ============================================================
# YERLEŞTİRME SLOTLARI
# ============================================================
```

```
D2_X = 135.0 + X_Diff
D2_Y = 183.30 + Y_Diff
D2_Z = 6.5

D1_X = 136.0 + X_Diff
D1_Y = 203.30 + Y_Diff
D1_Z = 6.5

R1_X = 166.0 + X_Diff
R1_Y = 201.90 + Y_Diff
R1_Z = 4.5

R2_X = 165.50 + X_Diff
R2_Y = 181.70 + Y_Diff
R2_Z = 4.5

SLOT_COORDS = {
    "D1": (D1_X, D1_Y, D1_Z),
    "D2": (D2_X, D2_Y, D2_Z),
    "R1": (R1_X, R1_Y, R1_Z),
    "R2": (R2_X, R2_Y, R2_Z),
}

# ============================================================
# HAREKET HIZLARI
# ============================================================
FEED_XY = 3000
```

```
FEED_Z  = 600

# ============================================================
# BEKLEME SÜRELERİ
# ============================================================
PUMP_PICK_WAIT        = 2.0
AFTER_ROTATE_WAIT     = 1.0
AFTER_Z_MOVE_WAIT     = 5.0
TEST_SETTLE_WAIT      = 1.2
BEFORE_TEST_WAIT      = 0.3
AFTER_TEST_XY_WAIT    = 1.0
AFTER_TEST_READY_WAIT = 0.8

RELEASE_WAIT_AFTER_PUMP_OFF = 5.0
RELEASE_LIFT_Z              = 4.0
RELEASE_LIFT_WAIT           = 0.6

AFTER_SLOT_XY_WAIT          = 0.6
AFTER_SLOT_Z_DOWN_WAIT      = 0.3
SLOT_DWELL_MS_BEFORE_DROP   = 3000
PRINTER_FLUSH_BEFORE_DROP_S = 10.0

# ============================================================
# TEST RETRY
# ============================================================
TEST_RETRY_COUNT = 4
TEST_RETRY_WAIT  = 0.6
```

```
# ============================================================
# OKUNAMAYAN DEĞER İÇİN YENİDEN DENEME
# ============================================================
UNREADABLE_RETRY_COUNT = 4
UNREADABLE_WAIT_S = 15.0
USER_WARNING_TEXT = "LUTFEN DÜZGÜN YERLEŞTİRİN"

# ============================================================
# DİRENÇ TOLERANSI
# ============================================================
RESISTOR_TOLERANCE = 0.20

RESISTOR_TARGETS = {
    "223":  21400.0,
    "1001": 1000.0,
    "332":  3200.0,
    "331":  350.0,
}

# ============================================================
# ROTATE SONRASI XY KAYDIRMA
# ============================================================
ROTATED_X_OFFSET = 0
ROTATED_Y_OFFSET = 0

# ============================================================
```

```
# ADIM ADIM ONAY / DEBUG
# ============================================================
STEP_BY_STEP_CONFIRM = False

def wait_user(msg: str):
    if not STEP_BY_STEP_CONFIRM:
        return
    input(f"\n[ONAY BEKLENİYOR] {msg} -> Devam için Enter'a bas...")

# ============================================================
# ÖZEL HATA
# ============================================================
class UnreadableMeasurementError(Exception):
    pass

# ============================================================
# SERİ CİHAZ
# ============================================================
class SerialDevice:
    def __init__(self, port: str, baud: int, timeout: float = 1.0):
        self.port = port
        self.baud = baud
        self.timeout = timeout
        self.ser = None

    def open(self):
        self.ser = serial.Serial(self.port, self.baud, timeout=self.timeout)
```

```
        time.sleep(2.0)
        log_full(f"[SERIAL OPEN] port={self.port} baud={self.baud}")

    def close(self):
        if self.ser and self.ser.is_open:
            self.ser.close()
            log_full(f"[SERIAL CLOSE] port={self.port}")

    def write_line(self, text: str):
        if not self.ser or not self.ser.is_open:
            raise RuntimeError(f"Port açık değil: {self.port}")
        self.ser.write((text + "\n").encode("utf-8"))

    def write_raw(self, text: str):
        if not self.ser or not self.ser.is_open:
            raise RuntimeError(f"Port açık değil: {self.port}")
        self.ser.write(text.encode("utf-8"))

    def read_line(self) -> str:
        if not self.ser or not self.ser.is_open:
            raise RuntimeError(f"Port açık değil: {self.port}")
        return self.ser.readline().decode("utf-8", errors="ignore").strip()

# ============================================================
# 3D PRINTER
# ============================================================
class Printer:
```

```
def __init__(self, dev: SerialDevice):
    self.dev = dev

def connect(self):
    self.dev.open()
    time.sleep(0.5)
    self._drain_boot()
    self.send("G90")
    self.send("G21")
    self.send("M17")
    log_user("Printer hazır.")

def _drain_boot(self, max_lines: int = 25):
    for _ in range(max_lines):
        line = self.dev.read_line()
        if not line:
            break
        log_full(f"[PRINTER BOOT] {line}")

def flush_rx(self, duration_s: float = 0.5):
    log_full(f"[INFO] Printer RX flush: {duration_s:.2f}s")
    t0 = time.time()
    while time.time() - t0 < duration_s:
        line = self.dev.read_line()
        if line:
            log_full(f"[PRINTER RX FLUSH] {line}")
        else:
```

```
            time.sleep(0.02)

def send(self, gcode: str, timeout: float = None):
    if timeout is None:
        cmd = gcode.strip().split()[0].upper() if gcode.strip() else ""
        if cmd in ("G0", "G1"):
            timeout = 30.0
        elif cmd == "M400":
            timeout = 60.0
        elif cmd == "G4":
            timeout = 20.0
        else:
            timeout = 10.0

    log_full(f"[PRINTER TX] {gcode}")
    self.dev.write_line(gcode)

    t0 = time.time()
    while True:
        line = self.dev.read_line()
        if line:
            log_full(f"[PRINTER RX] {line}")
            ll = line.lower()
            if "ok" in ll:
                return
            if "error" in ll:
                raise RuntimeError(f"Printer error: {line}")
```

```
        if time.time() - t0 > timeout:
            raise TimeoutError(f"Printer timeout ({timeout}s): {gcode}")

    def wait_all(self):
        self.send("M400")

    def dwell_ms(self, ms: int):
        self.send(f"G4 P{int(ms)}")

    def move_xy(self, x: float, y: float, feed: int = FEED_XY):
        self.send(f"G0 X{x:.3f} Y{y:.3f} F{feed}")

    def move_z(self, z: float, feed: int = FEED_Z):
        self.send(f"G0 Z{z:.3f} F{feed}")

    def go_safe_only(self):
        log_full(f"[INFO] SAFE noktasına gidiliyor: Z={SAFE_Z:.2f}")
        log_user("Güvenli yüksekliğe çıkılıyor.")
        self.move_z(SAFE_Z)
        self.wait_all()
        time.sleep(0.5)

    def go_home(self):
        log_full(f"[INFO] HOME noktasına gidiliyor: X={HOME_X:.2f}, Y={HOME_Y:.2f},
Z={SAFE_Z:.2f}")
        log_user("HOME noktasına gidiliyor.")
```

```
        self.move_z(SAFE_Z)
        self.wait_all()
        self.move_xy(HOME_X, HOME_Y)
        self.wait_all()
        time.sleep(0.5)

# ============================================================
# ARDUINO
# ============================================================
class ArduinoActuator:
    def __init__(self, dev: SerialDevice):
        self.dev = dev

    def connect(self):
        self.dev.open()
        time.sleep(0.5)
        self._drain_boot()
        log_user("Arduino hazır.")

    def _drain_boot(self, max_lines: int = 20):
        for _ in range(max_lines):
            line = self.dev.read_line()
            if not line:
                break
            log_full(f"[ARDUINO BOOT] {line}")

    def _send_char(self, c: str, wait_s: float):
```

```
    log_full(f"[ARDUINO TX] {c}")
    self.dev.write_raw(c)
    t0 = time.time()
    while time.time() - t0 < wait_s:
        line = self.dev.read_line()
        if line:
            log_full(f"[ARDUINO RX] {line}")

def pump_on(self):
    log_full("[INFO] Vakum açılıyor...")
    log_user("Vakum açılıyor.")
    self._send_char("P", wait_s=0.6)
    time.sleep(PUMP_PICK_WAIT)

def pump_force_on(self):
    log_full("[INFO] Vakum FORCE-ON (P) gönderiliyor...")
    self._send_char("P", wait_s=0.2)

def pump_off(self):
    log_full("[INFO] Vakum kapatılıyor...")
    log_user("Vakum kapatılıyor.")
    self._send_char("O", wait_s=0.6)
    time.sleep(0.2)

def rotate_fwd_180(self):
    log_full("[INFO] Diyot 180 derece çevriliyor...")
    log_user("Diyot yönü ters, 180 derece çevriliyor.")
```

```
        self._send_char("F", wait_s=4.2)
        time.sleep(AFTER_ROTATE_WAIT)

    def rotate_back_180(self):
        log_full("[INFO] Mekanizma eski yönüne alınıyor...")
        self._send_char("B", wait_s=4.2)
        time.sleep(AFTER_ROTATE_WAIT)

# ============================================================
# NANO DİYOT TEST
# ============================================================
class DiodeTester:
    def __init__(self, dev: SerialDevice):
        self.dev = dev

    def connect(self):
        self.dev.open()
        time.sleep(0.5)
        self._drain_boot()
        log_user("Nano hazır.")

    def _drain_boot(self, max_lines: int = 30):
        for _ in range(max_lines):
            line = self.dev.read_line()
            if not line:
                break
            log_full(f"[NANO BOOT] {line}")
```

```
    def test_once(self, timeout: float = 2.5):
        log_full("[NANO TX] d")
        self.dev.write_raw("d")

        lines = []
        t0 = time.time()
        while time.time() - t0 < timeout:
            line = self.dev.read_line()
            if line:
                log_full(f"[NANO RX] {line}")
                lines.append(line)
            else:
                time.sleep(0.03)

        for line in lines:
            if line.startswith("D;"):
                parsed = parse_diode_line(line)
                if parsed is not None:
                    return parsed

        raise RuntimeError("Nano diyot çıktısı parse edilemedi")

    def test_with_retry(self, retry_count: int = TEST_RETRY_COUNT, wait_s: float =
TEST_RETRY_WAIT):
        last_err = None
        for i in range(retry_count):
```

```
        try:
            log_full(f"[NANO] diyot test denemesi {i+1}/{retry_count}")
            result = self.test_once()
            if is_valid_test_result(result):
                return result
            log_full("[NANO] Geçersiz diyot çıktısı, tekrar...")
        except Exception as e:
            last_err = e
            log_full(f"[NANO] Diyot test hatası: {e}")
        time.sleep(wait_s)
    raise RuntimeError(f"Nano diyot testi başarısız. Son hata: {last_err}")

# ============================================================
# NANO DİRENÇ TEST
# ============================================================
class ResistorTester:
    def __init__(self, dev: SerialDevice):
        self.dev = dev

    def connect(self):
        pass

    def test_once(self, timeout: float = 3.0):
        log_full("[NANO TX] r")
        self.dev.write_raw("r")

        lines = []
```

```
        t0 = time.time()
        while time.time() - t0 < timeout:
            line = self.dev.read_line()
            if line:
                log_full(f"[NANO RX] {line}")
                lines.append(line)
            else:
                time.sleep(0.03)

        for line in lines:
            ohm = parse_resistor_ohm(line)
            if ohm is not None:
                return {"ohm": ohm, "raw": line}

        raise RuntimeError("Nano direnç çıktısı parse edilemedi")

    def test_with_retry(self, retry_count: int = TEST_RETRY_COUNT, wait_s: float =
TEST_RETRY_WAIT):
        last_err = None
        for i in range(retry_count):
            try:
                log_full(f"[NANO] direnç test denemesi {i+1}/{retry_count}")
                result = self.test_once()
                if result["ohm"] > 0:
                    return result
                log_full("[NANO] Geçersiz direnç çıktısı, tekrar...")
            except Exception as e:
```

```
            last_err = e
            log_full(f"[NANO] Direnç test hatası: {e}")
        time.sleep(wait_s)
    raise RuntimeError(f"Nano direnç testi başarısız. Son hata: {last_err}")


# ============================================================
# PARSE / KARAR
# ============================================================
def parse_diode_line(line: str):
    m = re.search(r"V1=(\d+);V2=(\d+);DIFF=(\d+);RES=(.+)$", line)
    if not m:
        return None
    return {
        "v1": int(m.group(1)),
        "v2": int(m.group(2)),
        "diff": int(m.group(3)),
        "res": m.group(4).strip(),
        "raw": line
    }


def parse_resistor_ohm(line: str):
    patterns = [
        r"RX\s*=\s*([0-9]+(?:\.[0-9]+)?)",
        r"R\s*=\s*([0-9]+(?:\.[0-9]+)?)",
        r"R\s*:\s*([0-9]+(?:\.[0-9]+)?)",
        r"RES\s*=\s*([0-9]+(?:\.[0-9]+)?)",
        r"OHM\s*=\s*([0-9]+(?:\.[0-9]+)?)",
```

```
        r"([0-9]+(?:\.[0-9]+)?)\s*OHM",
        r"^([0-9]+(?:\.[0-9]+)?)$",
    ]
    for pat in patterns:
        m = re.search(pat, line, flags=re.IGNORECASE)
        if m:
            return float(m.group(1))
    return None

def is_valid_test_result(test_result: dict) -> bool:
    if not test_result:
        return False

    res = test_result.get("res", "")
    v1 = test_result.get("v1", -1)
    v2 = test_result.get("v2", -1)
    diff = test_result.get("diff", -1)

    if not (("FWD_TEST1" in res) or ("FWD_TEST2" in res)):
        return False

    if v1 < 0 or v2 < 0 or diff < 0:
        return False

    if diff < 2000:
        return False
```

```
    return True

def direction_is_correct(test_result: dict) -> bool:
    return "FWD_TEST2" in test_result["res"]

def classify_diode(test_result: dict) -> str:
    v1   = test_result["v1"]
    v2   = test_result["v2"]
    diff = test_result["diff"]
    res  = test_result["res"]

    log_full(f"[CLASSIFY-DIODE] v1={v1}, v2={v2}, diff={diff}, res={res}")

    # ======================================================
    # DİYOT MERKEZ DEĞERLERİ
    # ======================================================
    # SK510C ölçümleri
    # FWD_TEST1:
    #   v1 ≈ 2732-2733
    #   diff ≈ 2732-2733
    # FWD_TEST2:
    #   v2 ≈ 2243-2277
    #   diff ≈ 2719-2752

    # S3M ölçümleri
    # FWD_TEST1:
    #   v1 ≈ 2903-2925
```

```
    #   diff ≈ 2903-2925
    # FWD_TEST2:
    #   v2 ≈ 2141-2164
    #   diff ≈ 2831-2854

    SK_T1_V1_CENTER   = 2733
    SK_T1_DIFF_CENTER = 2733

    S3M_T1_V1_CENTER   = 2915
    S3M_T1_DIFF_CENTER = 2915

    SK_T2_V2_CENTER   = 2257
    SK_T2_DIFF_CENTER = 2735

    S3M_T2_V2_CENTER   = 2150
    S3M_T2_DIFF_CENTER = 2842

    # Skorlar çok yakınsa yanlış karar vermek yerine retry'e düşsün
    UNSURE_GAP = 10

    if "FWD_TEST1" in res:
        # TEST1 için v1 + diff çok iyi ayırıyor
        sk_score  = abs(v1 - SK_T1_V1_CENTER)  + abs(diff - SK_T1_DIFF_CENTER)
        s3m_score = abs(v1 - S3M_T1_V1_CENTER) + abs(diff - S3M_T1_DIFF_CENTER)

        log_full(f"[CLASSIFY-DIODE] TEST1 skor -> SK510C={sk_score},
S3M={s3m_score}")
```

```
    elif "FWD_TEST2" in res:
        # TEST2 için v1 ayırıcı değil, o yüzden v2 + diff kullanıyoruz
        sk_score  = abs(v2 - SK_T2_V2_CENTER)  + abs(diff - SK_T2_DIFF_CENTER)
        s3m_score = abs(v2 - S3M_T2_V2_CENTER) + abs(diff - S3M_T2_DIFF_CENTER)

        log_full(f"[CLASSIFY-DIODE] TEST2 skor -> SK510C={sk_score},
S3M={s3m_score}")

    else:
        raise RuntimeError(f"Sınıflandırma için yön bilgisi yok: {res}")

    score_gap = abs(sk_score - s3m_score)
    if score_gap < UNSURE_GAP:
        raise RuntimeError(
            f"Diyot tipi sınır bölgede kaldı. "
            f"SK510C skor={sk_score}, S3M skor={s3m_score}, fark={score_gap}"
        )

    diode_type = "SK510C" if sk_score < s3m_score else "S3M"

    log_full(f"[INFO] Diyot tipi belirlendi: {diode_type}")
    log_user(f"Diyot tipi ölçüldü: {diode_type}")
    return diode_type

def resistor_in_range(measured_ohm: float, nominal_ohm: float, tol: float =
RESISTOR_TOLERANCE) -> bool:
    low = nominal_ohm * (1.0 - tol)
```

```
    high = nominal_ohm * (1.0 + tol)
    return low <= measured_ohm <= high

def classify_resistor(measured_ohm: float):
    log_full(f"[CLASSIFY-RES] measured={measured_ohm:.2f} ohm")
    matches = []
    for code, nominal in RESISTOR_TARGETS.items():
        if resistor_in_range(measured_ohm, nominal):
            matches.append(code)

    if len(matches) == 1:
        log_full(f"[INFO] Direnç tipi belirlendi: {matches[0]}")
        log_user(f"Direnç ölçüldü: {matches[0]}")
        return matches[0]
    if len(matches) > 1:
        raise RuntimeError(f"Direnç birden fazla tipe uydu: {matches}")
    raise RuntimeError(f"Direnç değeri hiçbir hedefe uymadı: {measured_ohm:.2f} ohm")

def apply_rotation_xy_offset(x: float, y: float, offset_active: bool):
    if offset_active:
        return x + ROTATED_X_OFFSET, y + ROTATED_Y_OFFSET
    return x, y

# ============================================================
# KULLANICI MESAJI
# ============================================================
def show_user_warning(msg: str):
```

```
    log_user(f"UYARI: {msg}")
    log_full("\n" + "=" * 60)
    log_full(f"[KULLANICI UYARISI] {msg}")
    log_full("=" * 60 + "\n")

# ============================================================
# YARDIMCI AKIŞLAR
# ============================================================
def return_to_pick_and_wait_for_reseat(printer: Printer, arduino: ArduinoActuator,
                                       px: float, py: float, pz: float,
                                       offset_active: bool,
                                       wait_s: float = UNREADABLE_WAIT_S):
    wait_x, wait_y = apply_rotation_xy_offset(px, py, offset_active)

    log_full("[INFO] Ölçüm okunamadı. Parça vakum açık şekilde pick lokasyonuna geri
götürülüyor...")
    log_user("Ölçüm okunamadı, parça düzeltme için geri götürülüyor.")
    arduino.pump_force_on()

    printer.go_safe_only()
    printer.move_xy(wait_x, wait_y)
    printer.wait_all()

    log_full("[INFO] Z havada tutuluyor, kullanıcı düzeltmesi için aşağı inilmiyor.")
    arduino.pump_force_on()

    show_user_warning(USER_WARNING_TEXT)
```

```
    log_full(f"[INFO] Kullanıcı müdahalesi için {wait_s:.1f} sn bekleniyor...")
    time.sleep(wait_s)

    log_full(f"[INFO] Bekleme bitti. Yeniden almak için aşağı iniliyor: Z={pz:.2f}")
    arduino.pump_force_on()
    printer.move_z(pz)
    printer.wait_all()
    time.sleep(AFTER_Z_MOVE_WAIT)

    log_full("[INFO] Parça tekrar alınsın diye kısa süre bekleniyor...")
    time.sleep(PUMP_PICK_WAIT)

    log_full(f"[INFO] Tekrar SAFE yüksekliğine çıkılıyor: Z={SAFE_Z:.2f}")
    printer.move_z(SAFE_Z)
    printer.wait_all()
    time.sleep(AFTER_Z_MOVE_WAIT)

def drop_back_to_pick_and_release(printer: Printer, arduino: ArduinoActuator,
                                  px: float, py: float, pz: float,
                                  offset_active: bool):
    drop_x, drop_y = apply_rotation_xy_offset(px, py, offset_active)

    log_full("[INFO] Parça pick noktasına geri bırakılıyor...")
    log_user("Parça geri bırakılıyor.")
    printer.go_safe_only()
    printer.move_xy(drop_x, drop_y)
    printer.wait_all()
```

```
    log_full(f"[INFO] Geri bırakmak için aşağı iniliyor: Z={pz:.2f}")
    printer.move_z(pz)
    printer.wait_all()
    time.sleep(AFTER_Z_MOVE_WAIT)

    arduino.pump_off()
    time.sleep(RELEASE_WAIT_AFTER_PUMP_OFF)

    printer.move_z(min(pz + 1.5, SAFE_Z))
    printer.wait_all()
    time.sleep(RELEASE_LIFT_WAIT)

    printer.move_z(SAFE_Z)
    printer.wait_all()
    time.sleep(AFTER_Z_MOVE_WAIT)

def move_to_diode_test_station(printer: Printer, arduino: ArduinoActuator, offset_active:
bool = False):
    arduino.pump_force_on()

    tx, ty = apply_rotation_xy_offset(DIODE_TEST_X, DIODE_TEST_Y, offset_active)
    log_full(f"[INFO] Diyot test istasyonuna gidiliyor: X={tx:.2f}, Y={ty:.2f}")
    log_user("Diyot test istasyonuna gidiliyor.")
    printer.move_xy(tx, ty)
    printer.wait_all()
    time.sleep(AFTER_TEST_XY_WAIT)
```

```
    arduino.pump_force_on()

    log_full(f"[INFO] Diyot test istasyonuna iniliyor: Z={DIODE_TEST_Z:.2f}")
    printer.move_z(DIODE_TEST_Z)
    printer.wait_all()
    time.sleep(AFTER_Z_MOVE_WAIT)
    time.sleep(TEST_SETTLE_WAIT)
    time.sleep(AFTER_TEST_READY_WAIT)
    time.sleep(BEFORE_TEST_WAIT)

def move_to_resistor_test_station(printer: Printer, arduino: ArduinoActuator):
    arduino.pump_force_on()

    log_full(f"[INFO] Direnç test istasyonuna gidiliyor: X={RES_TEST_X:.2f},
Y={RES_TEST_Y:.2f}")
    log_user("Direnç test istasyonuna gidiliyor.")
    printer.move_xy(RES_TEST_X, RES_TEST_Y)
    printer.wait_all()
    time.sleep(AFTER_TEST_XY_WAIT)

    arduino.pump_force_on()

    log_full(f"[INFO] Direnç test istasyonuna iniliyor: Z={RES_TEST_Z:.2f}")
    printer.move_z(RES_TEST_Z)
    printer.wait_all()
    time.sleep(AFTER_Z_MOVE_WAIT)
```

```
    time.sleep(TEST_SETTLE_WAIT)
    time.sleep(AFTER_TEST_READY_WAIT)
    time.sleep(BEFORE_TEST_WAIT)

# ============================================================
# KULLANICI MENÜSÜ
# ============================================================
def choose_mode():
    while True:
        print("\n=== MOD SEÇİMİ ===")
        print("1 -> 2 diyot + 2 direnç")
        print("2 -> 4 direnç")
        print("q -> çıkış")
        s = input("Seçim: ").strip().lower()
        if s in ("1", "2", "q"):
            return s
        print("Geçersiz seçim.")

def choose_from_list(prompt, options):
    while True:
        print(f"\n{prompt}")
        for i, item in enumerate(options, start=1):
            print(f"{i} -> {item}")
        s = input("Seçim: ").strip()
        if not s.isdigit():
            print("Geçersiz seçim.")
            continue
```

```
        idx = int(s) - 1
        if 0 <= idx < len(options):
            return options[idx]
        print("Geçersiz seçim.")

def get_diode_slot_mapping():
    print("\n=== DİYOT SLOT EŞLEŞMESİ ===")
    d1_choice = choose_from_list("D1 slotuna hangi diyot gitsin?", ["S3M", "SK510C"])
    d2_choice = "SK510C" if d1_choice == "S3M" else "S3M"
    print(f"D2 slotuna otomatik {d2_choice} atanacak.")
    return {
        d1_choice: "D1",
        d2_choice: "D2",
    }

def get_resistor_value_to_slot_mapping_mode1():
    print("\n=== MOD 1 DİRENÇ EŞLEŞMESİ ===")
    available = ["223", "331", "1001", "332"]
    mapping = {}

    choice_r1 = choose_from_list("R1 slotuna hangi direnç değeri gitsin?", available)
    mapping[choice_r1] = "R1"
    available.remove(choice_r1)

    choice_r2 = choose_from_list("R2 slotuna hangi direnç değeri gitsin?", available)
    mapping[choice_r2] = "R2"
    available.remove(choice_r2)
```

```
    return mapping

def get_resistor_value_to_slot_mapping_mode2():
    print("\n=== MOD 2 DİRENÇ EŞLEŞMESİ ===")
    available = ["223", "331", "1001", "332"]
    mapping = {}

    for slot in ["D1", "D2", "R1", "R2"]:
        choice = choose_from_list(f"{slot} slotuna hangi direnç değeri gitsin?", available)
        mapping[choice] = slot
        available.remove(choice)

    return mapping

# ============================================================
# ORTAK BIRAKMA
# ============================================================
def place_component(printer: Printer, arduino: ArduinoActuator, slot_name: str,
offset_active: bool = False):
    tx, ty, tz = SLOT_COORDS[slot_name]
    tx, ty = apply_rotation_xy_offset(tx, ty, offset_active)

    log_full(f"[INFO] {slot_name} slotuna gidiliyor: X={tx:.2f}, Y={ty:.2f}, Z={tz:.2f} |
offset_active={offset_active}")
    log_user(f"{slot_name} slotuna yerleştirme yapılıyor.")

    arduino.pump_force_on()
```

```
printer.go_safe_only()
printer.move_xy(tx, ty)
printer.wait_all()
time.sleep(AFTER_SLOT_XY_WAIT)

arduino.pump_force_on()

log_full(f"[INFO] Slot yüksekliğine iniliyor: Z={tz:.2f}")
printer.move_z(tz)
printer.wait_all()
time.sleep(AFTER_SLOT_Z_DOWN_WAIT)

log_full(f"[INFO] Slotta sabit bekleniyor: {SLOT_DWELL_MS_BEFORE_DROP} ms")
printer.dwell_ms(SLOT_DWELL_MS_BEFORE_DROP)

printer.wait_all()
printer.flush_rx(PRINTER_FLUSH_BEFORE_DROP_S)

arduino.pump_off()
log_full("[INFO] Vakum kapandı, parçanın ayrılması bekleniyor...")
time.sleep(RELEASE_WAIT_AFTER_PUMP_OFF)

first_lift_z = min(tz + RELEASE_LIFT_Z, SAFE_Z)
log_full(f"[INFO] İlk güvenli kalkış: Z={first_lift_z:.2f}")
printer.move_z(first_lift_z)
printer.wait_all()
```

```
    time.sleep(RELEASE_LIFT_WAIT)

    printer.move_z(SAFE_Z)
    printer.wait_all()
    time.sleep(AFTER_Z_MOVE_WAIT)

    log_full("[INFO] Parça bırakma tamamlandı.")
    log_user(f"{slot_name} slotuna yerleştirme tamamlandı.")

# ============================================================
# DİYOT İŞLEME
# ============================================================
def process_one_diode(printer: Printer, arduino: ArduinoActuator, diode_tester: DiodeTester,
pick_tuple, diode_slot_map):
    pick_name, px, py, pz = pick_tuple
    log_full(f"\n===== DİYOT İŞLENİYOR: {pick_name} =====")
    log_user(f"{pick_name} alınıyor.")

    did_rotate = False
    diode_type = None
    rotation_offset_active = False

    try:
        printer.go_safe_only()
        printer.move_xy(px, py)
        printer.wait_all()
```

```
        log_full(f"[INFO] Diyot alma noktasına iniliyor: Z={pz:.2f}")
        printer.move_z(pz)
        printer.wait_all()
        time.sleep(AFTER_Z_MOVE_WAIT)

        arduino.pump_on()

        log_full(f"[INFO] SAFE'e geri çıkılıyor: Z={SAFE_Z:.2f}")
        printer.move_z(SAFE_Z)
        printer.wait_all()
        time.sleep(AFTER_Z_MOVE_WAIT)

        unreadable_success = False

        for unreadable_try in range(1, UNREADABLE_RETRY_COUNT + 1):
            try:
                log_full(f"\n[INFO] Diyot okuma çevrimi
{unreadable_try}/{UNREADABLE_RETRY_COUNT}")
                log_user(f"{pick_name} test ediliyor. Deneme
{unreadable_try}/{UNREADABLE_RETRY_COUNT}")

                printer.go_safe_only()
                move_to_diode_test_station(printer, arduino, offset_active=rotation_offset_active)

                test = diode_tester.test_with_retry()
                log_full(f"[DIODE TEST RESULT] {test}")

                if direction_is_correct(test):
```

```
            log_user("Diyot yönü doğru.")
            diode_type = classify_diode(test)
            unreadable_success = True
            break

        log_full("[INFO] Diyot ters algılandı, SAFE'e çıkılıp çevrilecek.")
        log_user("Diyot yönü ters algılandı.")
        printer.move_z(SAFE_Z)
        printer.wait_all()
        time.sleep(AFTER_Z_MOVE_WAIT)

        arduino.rotate_fwd_180()
        did_rotate = True
        rotation_offset_active = True

        log_full(f"[INFO] Rotate oldu. Offset: X={ROTATED_X_OFFSET},
Y={ROTATED_Y_OFFSET}")

        printer.go_safe_only()
        move_to_diode_test_station(printer, arduino, offset_active=True)

        test = diode_tester.test_with_retry()
        log_full(f"[DIODE RETEST RESULT] {test}")

        if not direction_is_correct(test):
            raise UnreadableMeasurementError("Döndürmeden sonra da diyot yönü
okunamadı/doğrulanamadı")
```

```
        log_user("Döndürme sonrası yön doğru.")
        diode_type = classify_diode(test)
        unreadable_success = True
        break

    except Exception as e:
        log_full(f"[WARN] Diyot ölçümü okunamadı / doğrulanamadı: {e}")

        if did_rotate:
            log_full("[INFO] Yeniden deneme öncesi mekanizma eski yönüne alınıyor...")
            printer.go_safe_only()
            arduino.rotate_back_180()
            did_rotate = False
            rotation_offset_active = False

        if unreadable_try < UNREADABLE_RETRY_COUNT:
            return_to_pick_and_wait_for_reseat(
                printer=printer,
                arduino=arduino,
                px=px,
                py=py,
                pz=pz,
                offset_active=False,
                wait_s=UNREADABLE_WAIT_S
            )
        else:
```

```
            log_full(f"[WARN] Diyot {pick_name} için
{UNREADABLE_RETRY_COUNT} denemede de geçerli okuma alınamadı.")
            log_user(f"{pick_name} okunamadı, geri bırakılıyor.")
            drop_back_to_pick_and_release(
                printer=printer,
                arduino=arduino,
                px=px,
                py=py,
                pz=pz,
                offset_active=False
            )
            return None

    if not unreadable_success or diode_type is None:
        log_full(f"[WARN] {pick_name} atlanıyor.")
        return None

    log_full("[INFO] Test bitti, SAFE'e çıkılıyor.")
    printer.move_z(SAFE_Z)
    printer.wait_all()
    time.sleep(AFTER_Z_MOVE_WAIT)

    arduino.pump_force_on()
    target_slot = diode_slot_map[diode_type]
    log_full(f"[INFO] {diode_type} -> {target_slot}")
    log_user(f"{diode_type} tipi {target_slot} slotuna gönderiliyor.")
```

```
    place_component(printer, arduino, target_slot, offset_active=rotation_offset_active)

except Exception as e:
    log_full(f"[DIODE FAIL] {pick_name}: {e}")
    log_user(f"{pick_name} işlenirken hata oluştu.")
    try:
        drop_back_to_pick_and_release(
            printer=printer,
            arduino=arduino,
            px=px,
            py=py,
            pz=pz,
            offset_active=rotation_offset_active
        )
    except Exception as drop_err:
        log_full(f"[WARN] Diyot geri bırakma sırasında hata: {drop_err}")
    diode_type = None

finally:
    if did_rotate:
        try:
            printer.go_safe_only()
            arduino.rotate_back_180()
        except Exception as rot_err:
            log_full(f"[WARN] rotate_back_180 hatası: {rot_err}")

return diode_type
```

```
# ============================================================
# DİRENÇ İŞLEME
# ============================================================
def process_one_resistor(printer: Printer, arduino: ArduinoActuator, resistor_tester: ResistorTester,
                         pick_tuple, resistor_value_to_slot_map):
    pick_name, px, py, pz = pick_tuple
    log_full(f"\n===== DİRENÇ İŞLENİYOR: {pick_name} =====")
    log_user(f"{pick_name} alınıyor.")

    identified_code = None
    target_slot = None

    try:
        printer.go_safe_only()
        printer.move_xy(px, py)
        printer.wait_all()

        log_full(f"[INFO] Direnç alma noktasına iniliyor: Z={pz:.2f}")
        printer.move_z(pz)
        printer.wait_all()
        time.sleep(AFTER_Z_MOVE_WAIT)

        arduino.pump_on()

        log_full(f"[INFO] SAFE'e geri çıkılıyor: Z={SAFE_Z:.2f}")
```

```
        printer.move_z(SAFE_Z)
        printer.wait_all()
        time.sleep(AFTER_Z_MOVE_WAIT)

        unreadable_success = False

        for unreadable_try in range(1, UNREADABLE_RETRY_COUNT + 1):
            try:
                log_full(f"\n[INFO] Direnç okuma çevrimi {unreadable_try}/{UNREADABLE_RETRY_COUNT}")
                log_user(f"{pick_name} test ediliyor. Deneme {unreadable_try}/{UNREADABLE_RETRY_COUNT}")

                printer.go_safe_only()
                move_to_resistor_test_station(printer, arduino)

                test = resistor_tester.test_with_retry()
                measured_ohm = test["ohm"]
                log_full(f"[RES TEST RESULT] {measured_ohm:.2f} ohm | raw={test['raw']}")

                identified_code = classify_resistor(measured_ohm)

                if identified_code not in resistor_value_to_slot_map:
                    raise UnreadableMeasurementError(
                        f"Ölçülen direnç değeri ({identified_code}) kullanıcı eşleşmesinde yok"
                    )

                target_slot = resistor_value_to_slot_map[identified_code]
```

```
            unreadable_success = True
            break

        except Exception as e:
            log_full(f"[WARN] Direnç ölçümü okunamadı / doğrulanamadı: {e}")

            if unreadable_try < UNREADABLE_RETRY_COUNT:
                return_to_pick_and_wait_for_reseat(
                    printer=printer,
                    arduino=arduino,
                    px=px,
                    py=py,
                    pz=pz,
                    offset_active=False,
                    wait_s=UNREADABLE_WAIT_S
                )
            else:
                log_full(f"[WARN] Direnç {pick_name} için
{UNREADABLE_RETRY_COUNT} denemede de geçerli okuma alınamadı.")
                log_user(f"{pick_name} okunamadı, geri bırakılıyor.")
                drop_back_to_pick_and_release(
                    printer=printer,
                    arduino=arduino,
                    px=px,
                    py=py,
                    pz=pz,
                    offset_active=False
```

```
            )
            return None, None

    if not unreadable_success or identified_code is None or target_slot is None:
        log_full(f"[WARN] {pick_name} atlanıyor.")
        return None, None

    log_full("[INFO] Test bitti, SAFE'e çıkılıyor.")
    printer.move_z(SAFE_Z)
    printer.wait_all()
    time.sleep(AFTER_Z_MOVE_WAIT)

    arduino.pump_force_on()
    log_full(f"[INFO] Ölçülen değer {identified_code} -> hedef slot {target_slot}")
    log_user(f"{identified_code} kodlu direnç {target_slot} slotuna gönderiliyor.")
    place_component(printer, arduino, target_slot, offset_active=False)

except Exception as e:
    log_full(f"[RES FAIL] {pick_name}: {e}")
    log_user(f"{pick_name} işlenirken hata oluştu.")
    try:
        drop_back_to_pick_and_release(
            printer=printer,
            arduino=arduino,
            px=px,
            py=py,
            pz=pz,
```

```
            offset_active=False
        )
    except Exception as drop_err:
        log_full(f"[WARN] Direnç geri bırakma sırasında hata: {drop_err}")
    identified_code = None
    target_slot = None

return identified_code, target_slot

# ============================================================
# ANA AKIŞLAR
# ============================================================
def run_mode_1(printer, arduino, diode_tester, resistor_tester):
    log_full("\n######## MOD 1: 2 DİYOT + 2 DİRENÇ ########")
    diode_slot_map = get_diode_slot_mapping()
    resistor_value_to_slot_map = get_resistor_value_to_slot_mapping_mode1()

    results = {
        "diodes": [],
        "resistors": []
    }

    for pick in DIODE_PICK_LIST:
        dtype = process_one_diode(printer, arduino, diode_tester, pick, diode_slot_map)
        results["diodes"].append((pick[0], dtype))

    for pick in RESISTOR_PICK_LIST[:2]:
```

```
        identified_code, slot = process_one_resistor(
            printer, arduino, resistor_tester,
            pick_tuple=pick,
            resistor_value_to_slot_map=resistor_value_to_slot_map
        )
        results["resistors"].append((pick[0], identified_code, slot))

    return results

def run_mode_2(printer, arduino, resistor_tester):
    log_full("\n######## MOD 2: 4 DİRENÇ ########")
    resistor_value_to_slot_map = get_resistor_value_to_slot_mapping_mode2()

    results = {
        "resistors": []
    }

    for pick in RESISTOR_PICK_LIST:
        identified_code, slot = process_one_resistor(
            printer, arduino, resistor_tester,
            pick_tuple=pick,
            resistor_value_to_slot_map=resistor_value_to_slot_map
        )
        results["resistors"].append((pick[0], identified_code, slot))

    return results
```

```
# ============================================================
# KALİBRASYON MODU OKU
# ============================================================
def read_calibration_mode(mode: int):
    calib_path = get_placement_calib_json(mode)

    if not os.path.exists(calib_path):
        return None

    with open(calib_path, "r", encoding="utf-8") as f:
        data = json.load(f)

    return data.get("board_mode")

# ============================================================
# YERLEŞTİRME DOĞRULAMA ÇAĞIR
# ============================================================
def run_placement_verification(mode: int):
    requested_board_mode = "2d2r" if mode == 1 else "4r"
    calib_json_path = get_placement_calib_json(mode)

    log_full(f"\n[INFO] Yerleştirme doğrulama başlatılıyor...
requested_board_mode={requested_board_mode}")
    log_user("Yerleştirme doğrulaması başlatılıyor.")

    calib_mode = read_calibration_mode(mode)
    if calib_mode is None:
```

```
        raise RuntimeError(
            f"Kalibrasyon dosyası bulunamadı: {calib_json_path}\n"
            f"Önce şu komutla kalibrasyon yap:\n"
            f"{PLACEMENT_VERIFIER_PYTHON} {PLACEMENT_VERIFIER_PY} --mode calibrate --board-mode {requested_board_mode}"
        )

    log_full(f"[INFO] Kalibrasyon dosyasındaki mod: {calib_mode}")

    if calib_mode != requested_board_mode:
        raise RuntimeError(
            f"Kalibrasyon modu ile sistem modu farklı.\n"
            f"- Sistem şu an: {requested_board_mode}\n"
            f"- Kalibrasyon dosyası: {calib_mode}\n\n"
            f"Doğru çözüm: mevcut mod için yeniden kalibrasyon yap.\n"
            f"Örnek komut:\n"
            f"{PLACEMENT_VERIFIER_PYTHON} {PLACEMENT_VERIFIER_PY} --mode calibrate --board-mode {requested_board_mode}"
        )

    cmd = [
        PLACEMENT_VERIFIER_PYTHON,
        PLACEMENT_VERIFIER_PY,
        "--mode", "verify",
        "--board-mode", requested_board_mode,
        "--auto-capture",
    ]
```

```
log_full("[VERIFY CMD] " + " ".join(cmd))
proc = subprocess.run(cmd)
log_full(f"[VERIFY EXIT CODE] {proc.returncode}")

if proc.returncode != 0:
    raise RuntimeError(f"placement_verifier verify başarısız. exit_code={proc.returncode}")

if not os.path.exists(PLACEMENT_RESULT_JSON):
    raise RuntimeError(f"Doğrulama sonucu oluşmadı: {PLACEMENT_RESULT_JSON}")

with open(PLACEMENT_RESULT_JSON, "r", encoding="utf-8") as f:
    result = json.load(f)

log_full("\n========== YERLEŞTİRME DOĞRULAMA SONUCU ==========")
log_full(f"Overall Accuracy: %{result.get('overall_accuracy', 0.0)}")
log_user(f"Genel doğruluk: %{result.get('overall_accuracy', 0.0)}")

slots = result.get("slots", {})

if mode == 1:
    slot_order = ["D1", "D2", "R1", "R2"]
else:
    slot_order = ["D1", "D2", "R1", "R2"]

for slot_name in slot_order:
    if slot_name in slots:
        info = slots[slot_name]
```

```
        log_full(
            f"  - {slot_name}: "
            f"present={info.get('present')} | "
            f"status={info.get('status')} | "
            f"score=%{info.get('score')} | "
            f"pos_error_px={info.get('pos_error_px')} | "
            f"angle_error_deg={info.get('angle_error_deg')}"
        )

    log_full(f"[INFO] Debug görsel: {PLACEMENT_DEBUG_IMG}")
    log_user("Doğrulama görseli hazır.")
    return result

# ============================================================
# SONUÇ YAZDIR
# ============================================================
def print_summary(mode, results):
    log_full("\n================ SONUÇ ==================")
    log_full(f"Mod: {mode}")

    if "diodes" in results:
        log_full("\n[DİYOTLAR]")
        for pick_name, dtype in results["diodes"]:
            log_full(f"  - {pick_name}: {dtype}")

    if "resistors" in results:
        log_full("\n[DİRENÇLER]")
```

```
    for pick_name, identified, slot in results["resistors"]:
        log_full(f"  - {pick_name} -> Ölçülen: {identified} | Yerleşen slot: {slot}")

# ============================================================
# TEK ÇEVRİM ÇALIŞTIR
# ============================================================
def run_one_cycle(printer, arduino, diode_tester, resistor_tester, mode_str: str):
    mode = int(mode_str)
    results = {"diodes": [], "resistors": []}
    run_error = None

    log_full("[INFO] Başlangıç güvenliği: önce SAFE noktasına çıkılıyor...")
    try:
        printer.go_safe_only()
    except Exception as e:
        log_full(f"[WARN] SAFE'e çıkarken hata: {e}")

    try:
        if mode == 1:
            results = run_mode_1(printer, arduino, diode_tester, resistor_tester)
        else:
            results = run_mode_2(printer, arduino, resistor_tester)
    except Exception as e:
        run_error = e
        log_full(f"[RUN MODE ERROR] {e}")

    log_full("[INFO] Montaj sonrası doğrulama öncesi HOME'a gidilmeye çalışılıyor...")
```

```
    try:
        printer.go_home()
    except Exception as e:
        log_full(f"[WARN] HOME hatası: {e}")

    try:
        print_summary(mode, results)
    except Exception as e:
        log_full(f"[WARN] Summary hatası: {e}")

    log_full("[INFO] Şimdi yerleştirme doğrulaması başlatılacak...")
    placement_result = run_placement_verification(mode)
    log_full(f"[INFO] Final overall placement accuracy: %{placement_result.get('overall_accuracy', 0.0)}")

    if run_error is not None:
        raise RuntimeError(f"Montaj akışında hata vardı ama verify yine çalıştırıldı. Hata: {run_error}")

# ============================================================
# WEB'DEN ÇALIŞAN ÇEVRİM
# ============================================================
def run_one_cycle_web(printer, arduino, diode_tester, resistor_tester, config: dict):
    mode = int(config["mode"])

    results = {"diodes": [], "resistors": []}

    log_full("[INFO] Başlangıç güvenliği: önce SAFE noktasına çıkılıyor...")
```

```
try:
    printer.go_safe_only()
except Exception as e:
    log_full(f"[WARN] SAFE'e çıkarken hata: {e}")

if mode == 1:
    diode_slot_map = config["diode_slot_map"]
    resistor_value_to_slot_map = config["resistor_map_mode1"]

    log_user("Mod 1 başlatıldı: 2 diyot + 2 direnç")

    for pick in DIODE_PICK_LIST:
        dtype = process_one_diode(printer, arduino, diode_tester, pick, diode_slot_map)
        results["diodes"].append((pick[0], dtype))

    for pick in RESISTOR_PICK_LIST[:2]:
        identified_code, slot = process_one_resistor(
            printer, arduino, resistor_tester,
            pick_tuple=pick,
            resistor_value_to_slot_map=resistor_value_to_slot_map
        )
        results["resistors"].append((pick[0], identified_code, slot))

else:
    resistor_value_to_slot_map = config["resistor_map_mode2"]

    log_user("Mod 2 başlatıldı: 4 direnç")
```

```
        for pick in RESISTOR_PICK_LIST:
            identified_code, slot = process_one_resistor(
                printer, arduino, resistor_tester,
                pick_tuple=pick,
                resistor_value_to_slot_map=resistor_value_to_slot_map
            )
            results["resistors"].append((pick[0], identified_code, slot))

    log_full("[INFO] Montaj sonrası HOME'a gidilmeye çalışılıyor...")
    try:
        printer.go_home()
    except Exception as e:
        log_full(f"[WARN] HOME hatası: {e}")

    print_summary(mode, results)
    placement_result = run_placement_verification(mode)
    log_user(f"Yerleştirme doğrulaması tamamlandı. Accuracy =
%{placement_result.get('overall_accuracy', 0.0)}")

    return {
        "mode": mode,
        "results": results,
        "placement_result": placement_result
    }

# ============================================================
```

```
# MAIN
# ============================================================
def main():
    printer_dev = SerialDevice(PRINTER_PORT, PRINTER_BAUD, timeout=0.2)
    arduino_dev = SerialDevice(ARDUINO_PORT, ARDUINO_BAUD, timeout=1.0)
    nano_dev    = SerialDevice(NANO_PORT, NANO_BAUD, timeout=1.0)

    try:
        printer = Printer(printer_dev)
        arduino = ArduinoActuator(arduino_dev)
        diode_tester = DiodeTester(nano_dev)
        resistor_tester = ResistorTester(nano_dev)

        print("Cihazlara bağlanılıyor...")
        printer.connect()
        arduino.connect()
        diode_tester.connect()
        resistor_tester.connect()

        while True:
            secim = choose_mode()

            if secim == "q":
                print("\n[INFO] Kullanıcı çıkış istedi. Sistem kapatılıyor...")
                try:
                    printer.go_home()
                except Exception as e:
```

```
            print(f"[WARN] Home'a giderken hata: {e}")
        break

    try:
        run_one_cycle(printer, arduino, diode_tester, resistor_tester, secim)
        print("\n[INFO] İşlem tamamlandı. Menüye geri dönülüyor...")
    except Exception as e:
        print(f"\n[RUN HATASI] {e}")
        print("[INFO] Sistem güvenli konuma alınmaya çalışılıyor...")
        try:
            printer.go_safe_only()
            printer.go_home()
        except Exception as home_err:
            print(f"[WARN] Güvenli konuma dönüşte hata: {home_err}")

except KeyboardInterrupt:
    print("\nKullanıcı durdurdu.")
except Exception as e:
    print(f"\nHATA: {e}")
finally:
    try:
        printer_dev.close()
    except Exception:
        pass
    try:
        arduino_dev.close()
    except Exception:
```

```
        pass
    try:
        nano_dev.close()
    except Exception:
        pass

if __name__ == "__main__":
    main()
```

webui.py

```
#!/usr/bin/env python3
from flask import Flask, request, jsonify, send_file, render_template_string
import threading
import os
import traceback

import main_system as ms

app = Flask(__name__)

STATE = {
    "running": False,
    "full_logs": [],
    "user_logs": [],
    "current_config": None,
    "result_image_ready": False,
    "result_summary": None,
    "popup_open": False
```

```
}

LOCK = threading.Lock()

def add_full_log(msg: str):
    with LOCK:
        STATE["full_logs"].append(msg)
        STATE["full_logs"] = STATE["full_logs"][-800:]

def add_user_log(msg: str):
    with LOCK:
        STATE["user_logs"].append(msg)
        STATE["user_logs"] = STATE["user_logs"][-400:]

def reset_state_for_new_run(config):
    with LOCK:
        STATE["running"] = True
        STATE["full_logs"] = []
        STATE["user_logs"] = []
        STATE["current_config"] = config
        STATE["result_image_ready"] = False
        STATE["result_summary"] = None
        STATE["popup_open"] = False

def finish_state(summary=None):
    with LOCK:
        STATE["running"] = False
```

```
        STATE["result_summary"] = summary
        if os.path.exists(ms.PLACEMENT_DEBUG_IMG):
            STATE["result_image_ready"] = True
            STATE["popup_open"] = True

def validate_config(config):
    mode = int(config.get("mode", 0))
    if mode not in (1, 2):
        raise RuntimeError("Geçersiz mod.")

    valid_diodes = {"S3M", "SK510C"}
    valid_resistors = {"223", "331", "1001", "332"}

    if mode == 1:
        diode_map = config.get("diode_slot_map", {})
        resistor_map = config.get("resistor_map_mode1", {})

        if set(diode_map.keys()) != valid_diodes:
            raise RuntimeError("Mod 1 için diyot eşleşmesi eksik ya da hatalı.")

        if set(diode_map.values()) != {"D1", "D2"}:
            raise RuntimeError("Mod 1 için diyot slotları D1 ve D2 olmalı.")

        if len(resistor_map) != 2:
            raise RuntimeError("Mod 1 için 2 farklı direnç seçilmelidir.")

        if len(set(resistor_map.keys())) != 2:
```

```
            raise RuntimeError("Mod 1 için direnç değerleri tekrar etmemeli.")

        if not set(resistor_map.keys()).issubset(valid_resistors):
            raise RuntimeError("Mod 1 için geçersiz direnç kodu var.")

        if set(resistor_map.values()) != {"R1", "R2"}:
            raise RuntimeError("Mod 1 için direnç slotları R1 ve R2 olmalı.")

    elif mode == 2:
        resistor_map = config.get("resistor_map_mode2", {})

        if len(resistor_map) != 4:
            raise RuntimeError("Mod 2 için 4 farklı direnç seçilmelidir.")

        if set(resistor_map.keys()) != valid_resistors:
            raise RuntimeError("Mod 2 için tüm direnç kodları birer kez seçilmeli.")

        if set(resistor_map.values()) != {"D1", "D2", "R1", "R2"}:
            raise RuntimeError("Mod 2 için slotlar D1, D2, R1, R2 olmalı.")

def run_cycle_background(config):
    printer_dev = ms.SerialDevice(ms.PRINTER_PORT, ms.PRINTER_BAUD, timeout=0.2)
    arduino_dev = ms.SerialDevice(ms.ARDUINO_PORT, ms.ARDUINO_BAUD,
timeout=1.0)
    nano_dev    = ms.SerialDevice(ms.NANO_PORT, ms.NANO_BAUD, timeout=1.0)

    try:
```

```
    ms.set_log_callbacks(add_full_log, add_user_log)

    printer = ms.Printer(printer_dev)
    arduino = ms.ArduinoActuator(arduino_dev)
    diode_tester = ms.DiodeTester(nano_dev)
    resistor_tester = ms.ResistorTester(nano_dev)

    add_user_log("Cihazlara bağlanılıyor...")
    printer.connect()
    arduino.connect()
    diode_tester.connect()
    resistor_tester.connect()

    add_user_log("Sistem başlatıldı.")
    result = ms.run_one_cycle_web(
        printer=printer,
        arduino=arduino,
        diode_tester=diode_tester,
        resistor_tester=resistor_tester,
        config=config
    )

    add_user_log("İşlem tamamlandı.")
    finish_state(summary=result)

except Exception as e:
    add_full_log(f"[WEB RUN ERROR] {e}")
```

```
        add_full_log(traceback.format_exc())
        add_user_log(f"Hata oluştu: {e}")

        try:
            printer = ms.Printer(printer_dev)
            printer.go_safe_only()
            printer.go_home()
        except Exception:
            pass

        finish_state(summary={"error": str(e)})

    finally:
        try:
            printer_dev.close()
        except Exception:
            pass
        try:
            arduino_dev.close()
        except Exception:
            pass
        try:
            nano_dev.close()
        except Exception:
            pass

HTML_PAGE = """
```

```
<!doctype html>
<html lang="tr">
<head>
  <meta charset="utf-8">
  <title>ELE495 Web Arayüz</title>
  <style>
    :root {
      --bg: #0b1020;
      --panel: #121a2f;
      --card: #0f172a;
      --border: rgba(255,255,255,0.08);
      --text: #e5e7eb;
      --slot-border: #334155;
      --glow: 0 10px 30px rgba(0,0,0,0.35);
      --radius: 18px;
      --blue: #60a5fa;
      --green: #4ade80;
      --yellow: #fbbf24;
      --red: #ef4444;
      --userbg: #111a2e;
    }

    * { box-sizing: border-box; }

    body {
      margin: 0;
      font-family: Inter, "Segoe UI", Arial, sans-serif;
```

```
  background:
    radial-gradient(circle at top left, #16213e 0%, #0b1020 45%),
    linear-gradient(135deg, #0b1020, #111827);
  color: var(--text);
  height: 100vh;
  overflow: hidden;
}

.container {
  display: flex;
  gap: 14px;
  height: 100vh;
  padding: 14px;
}

.left {
  width: 31%;
  display: flex;
  flex-direction: column;
  gap: 14px;
}

.right {
  width: 69%;
  display: flex;
  flex-direction: column;
  gap: 14px;
```

```
}

.box {
  background: rgba(18, 26, 47, 0.92);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 18px;
  box-shadow: var(--glow);
  backdrop-filter: blur(8px);
  min-width: 0;
}

h2, h3 {
  margin-top: 0;
  margin-bottom: 14px;
  color: #f8fafc;
  letter-spacing: 0.2px;
}

.panel-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 12px;
}
```

```
.mini-btn {
  margin: 0;
  padding: 8px 12px;
  border: none;
  border-radius: 10px;
  cursor: pointer;
  font-size: 15px;
  font-weight: 700;
  color: white;
  background: linear-gradient(135deg, #334155, #1e293b);
  box-shadow: none;
}

.mini-btn:hover {
  filter: brightness(1.08);
}

.controls label {
  display: block;
  margin-top: 12px;
  margin-bottom: 6px;
  font-weight: bold;
  color: #dbeafe;
}

.controls select {
  width: 100%;
```

```
  padding: 12px 14px;
  border-radius: 12px;
  border: 1px solid #334155;
  background: #0f172a;
  color: #f8fafc;
  font-size: 16px;
  outline: none;
}

.controls select:focus {
  border-color: #60a5fa;
  box-shadow: 0 0 0 3px rgba(96,165,250,0.15);
}

button {
  padding: 13px 18px;
  border: none;
  background: linear-gradient(135deg, #2563eb, #3b82f6);
  color: white;
  border-radius: 14px;
  cursor: pointer;
  margin-top: 16px;
  font-size: 16px;
  font-weight: bold;
  box-shadow: 0 8px 20px rgba(37,99,235,0.35);
}
```

```
button:hover { filter: brightness(1.05); }

button:disabled {
  background: #475569;
  box-shadow: none;
  cursor: not-allowed;
}

.statusline {
  margin-top: 12px;
  font-weight: bold;
  font-size: 19px;
  color: #cbd5e1;
}

.status-running { color: var(--green); }
.status-idle { color: var(--yellow); }

.mode-warning {
  margin-top: 14px;
  padding: 12px 14px;
  border-radius: 12px;
  background: rgba(251,191,36,0.12);
  border: 1px solid rgba(251,191,36,0.32);
  color: #fde68a;
  font-size: 14px;
  line-height: 1.45;
```

```
  font-weight: 600;
}

.pcb-box {
  height: 52%;
  display: flex;
  flex-direction: column;
}

.pcb-panel {
  flex: 1;
  position: relative;
  overflow: hidden;
  border-radius: 22px;
  border: 1px solid rgba(255,255,255,0.08);
  background:
    linear-gradient(180deg, rgba(30,41,59,0.95), rgba(15,23,42,0.96)),
    linear-gradient(135deg, #0f172a, #1e293b);
  min-height: 360px;
}

.pcb-grid {
  position: absolute;
  inset: 0;
  background-image:
    linear-gradient(rgba(255,255,255,0.03) 1px, transparent 1px),
    linear-gradient(90deg, rgba(255,255,255,0.03) 1px, transparent 1px);
```

```
  background-size: 30px 30px;
  opacity: 0.35;
  pointer-events: none;
}

.pcb-title {
  position: absolute;
  top: 18px;
  left: 22px;
  font-size: 28px;
  font-weight: bold;
  color: #cbd5e1;
  letter-spacing: 1px;
}

.trace {
  position: absolute;
  background: linear-gradient(90deg, rgba(34,197,94,0.22), rgba(96,165,250,0.22));
  border-radius: 999px;
  border: 1px solid rgba(255,255,255,0.04);
}

.slot {
  position: absolute;
  width: 150px;
  min-height: 82px;
  border-radius: 18px;
```

```
  background:
    linear-gradient(180deg, rgba(30,41,59,0.96), rgba(15,23,42,0.98));
  border: 1px solid var(--slot-border);
  box-shadow:
    inset 0 0 0 1px rgba(255,255,255,0.03),
    0 10px 25px rgba(0,0,0,0.28);
  padding: 12px;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
}

.slot-name {
  font-size: 20px;
  font-weight: bold;
  color: #93c5fd;
  margin-bottom: 6px;
}

.slot-value {
  font-size: 20px;
  font-weight: bold;
  color: #f8fafc;
  text-align: center;
  word-break: break-word;
}
```

```
.slot.hint .slot-value {
  color: #64748b;
  font-weight: 600;
}

.slot.active {
  border-color: rgba(96,165,250,0.55);
  box-shadow:
    inset 0 0 0 1px rgba(255,255,255,0.03),
    0 0 0 1px rgba(96,165,250,0.18),
    0 10px 30px rgba(37,99,235,0.24);
}

.slot.diode {
  background:
    linear-gradient(180deg, rgba(36, 52, 77, 0.98), rgba(15, 23, 42, 0.98));
}

.slot.resistor {
  background:
    linear-gradient(180deg, rgba(42, 47, 74, 0.98), rgba(15, 23, 42, 0.98));
}

.bottom-row {
  display: flex;
  gap: 14px;
```

```
  height: 48%;
}

.half {
  width: 50%;
  display: flex;
  flex-direction: column;
  min-width: 0;
}

.userlogs {
  flex: 1;
  overflow-y: auto;
  background: var(--userbg);
  border-radius: 14px;
  padding: 10px;
  border: 1px solid #263244;
  font-size: 15px;
  line-height: 1.5;
}

.user-msg {
  background: linear-gradient(180deg, rgba(23,35,61,0.96), rgba(15,23,42,0.96));
  border: 1px solid rgba(96,165,250,0.12);
  color: #e2e8f0;
  border-radius: 12px;
  padding: 10px 12px;
```

```
  margin-bottom: 10px;
  box-shadow: inset 0 0 0 1px rgba(255,255,255,0.02);
  word-break: break-word;
}

.user-msg:last-child {
  margin-bottom: 0;
}

.logs {
  flex: 1;
  overflow-y: auto;
  white-space: pre-wrap;
  background: #020617;
  color: #4ade80;
  font-family: "Cascadia Code", Consolas, monospace;
  border-radius: 14px;
  padding: 12px;
  border: 1px solid #1e293b;
  font-size: 14px;
  line-height: 1.4;
}

.summary {
  background: #0f172a;
  border: 1px solid #263244;
  padding: 10px;
```

```
  border-radius: 12px;
  margin-top: 10px;
  white-space: pre-wrap;
  max-height: 140px;
  overflow-y: auto;
  color: #cbd5e1;
  font-size: 14px;
}

.modal {
  display: none;
  position: fixed;
  z-index: 9999;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  background: rgba(2,6,23,0.35);
  justify-content: center;
  align-items: flex-start;
  padding-top: 36px;
}

.modal-content {
  background: #0f172a;
  padding: 16px;
  border-radius: 18px;
```

```
  width: 78%;
  max-width: 1200px;
  box-shadow: 0 12px 40px rgba(0,0,0,0.4);
  border: 1px solid rgba(255,255,255,0.08);
}

.modal-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 12px;
}

.modal-content h3 {
  margin: 0;
  font-size: 28px;
  color: #f8fafc;
}

.modal-content img {
  width: 100%;
  max-height: 70vh;
  object-fit: contain;
  display: block;
  border-radius: 12px;
  border: 1px solid rgba(255,255,255,0.08);
  background: #020617;
```

```
}

.close-btn {
  margin-top: 0;
  padding: 8px 14px;
  border-radius: 12px;
  border: none;
  cursor: pointer;
  font-size: 20px;
  font-weight: bold;
  background: linear-gradient(135deg, #ef4444, #dc2626);
  color: white;
  box-shadow: 0 8px 18px rgba(239,68,68,0.28);
}

.close-btn:hover { filter: brightness(1.08); }

.panel-overlay {
  display: none;
  position: fixed;
  inset: 0;
  z-index: 9998;
  background: rgba(2,6,23,0.55);
  justify-content: center;
  align-items: center;
  padding: 22px;
}
```

```
    .panel-overlay-content {
      width: min(1200px, 95vw);
      height: min(82vh, 900px);
      background: #0f172a;
      border: 1px solid rgba(255,255,255,0.08);
      border-radius: 18px;
      box-shadow: 0 12px 40px rgba(0,0,0,0.45);
      display: flex;
      flex-direction: column;
      padding: 16px;
    }

    .panel-overlay-body {
      flex: 1;
      min-height: 0;
      overflow: auto;
    }

    .panel-overlay .userlogs,
    .panel-overlay .logs {
      height: 100%;
      flex: 1;
    }
  </style>
</head>
<body>
```

```
<div class="container">

  <div class="left">
    <div class="box">
      <h2>Kontrol Paneli</h2>

      <div class="controls">
        <label>Mod</label>
        <select id="mode" onchange="updateModeFields()">
          <option value="1">1 - 2 diyot + 2 direnç</option>
          <option value="2">2 - 4 direnç</option>
        </select>

        <div id="mode1-fields">
          <label>D1 slotuna gidecek diyot</label>
          <select id="d1_diode" onchange="updatePcbLabels()">
            <option value="">None</option>
            <option value="S3M">S3M</option>
            <option value="SK510C">SK510C</option>
          </select>

          <label>R1 slotuna gidecek direnç</label>
          <select id="r1_res" onchange="updatePcbLabels()">
            <option value="">None</option>
            <option value="223">223</option>
            <option value="331">331</option>
            <option value="1001">1001</option>
```

```
    <option value="332">332</option>
  </select>

  <label>R2 slotuna gidecek direnç</label>
  <select id="r2_res" onchange="updatePcbLabels()">
    <option value="">None</option>
    <option value="223">223</option>
    <option value="331">331</option>
    <option value="1001">1001</option>
    <option value="332">332</option>
  </select>

  <div id="mode1-warning" class="mode-warning">
    Lütfen seçtiğiniz dirençler Rp1 ve Rp2 yerine yerleştirin!
  </div>
</div>

<div id="mode2-fields" style="display:none;">
  <label>D1 slotuna gidecek direnç</label>
  <select id="m2_d1" onchange="updatePcbLabels()">
    <option value="">None</option>
    <option value="223">223</option>
    <option value="331">331</option>
    <option value="1001">1001</option>
    <option value="332">332</option>
  </select>
```

```
  <label>D2 slotuna gidecek direnç</label>
  <select id="m2_d2" onchange="updatePcbLabels()">
    <option value="">None</option>
    <option value="223">223</option>
    <option value="331">331</option>
    <option value="1001">1001</option>
    <option value="332">332</option>
  </select>

  <label>R1 slotuna gidecek direnç</label>
  <select id="m2_r1" onchange="updatePcbLabels()">
    <option value="">None</option>
    <option value="223">223</option>
    <option value="331">331</option>
    <option value="1001">1001</option>
    <option value="332">332</option>
  </select>

  <label>R2 slotuna gidecek direnç</label>
  <select id="m2_r2" onchange="updatePcbLabels()">
    <option value="">None</option>
    <option value="223">223</option>
    <option value="331">331</option>
    <option value="1001">1001</option>
    <option value="332">332</option>
  </select>
</div>
```

```
      <button id="startBtn" onclick="startSystem()">Sistemi Başlat</button>
      <div class="statusline" id="runState">Durum: Beklemede</div>
    </div>
  </div>
</div>

<div class="right">
  <div class="box pcb-box">
    <h2>PCB Yerleşim Paneli</h2>
    <div class="pcb-panel">
      <div class="pcb-grid"></div>
      <div class="pcb-title">PCB LAYOUT</div>

      <div class="trace" style="left: 31%; top: 33%; width: 38%; height: 8px;"></div>
      <div class="trace" style="left: 31%; top: 63%; width: 38%; height: 8px;"></div>
      <div class="trace" style="left: 28%; top: 36%; width: 8px; height: 110px;"></div>
      <div class="trace" style="left: 69%; top: 36%; width: 8px; height: 110px;"></div>

      <div id="slot_D1" class="slot diode hint" style="left: 18%; top: 22%;">
        <div class="slot-name">D1</div>
        <div class="slot-value">Seçim yok</div>
      </div>

      <div id="slot_D2" class="slot diode hint" style="left: 18%; top: 54%;">
        <div class="slot-name">D2</div>
        <div class="slot-value">Seçim yok</div>
```

```
    </div>

    <div id="slot_R1" class="slot resistor hint" style="left: 63%; top: 22%;">
      <div class="slot-name">R1</div>
      <div class="slot-value">Seçim yok</div>
    </div>

    <div id="slot_R2" class="slot resistor hint" style="left: 63%; top: 54%;">
      <div class="slot-name">R2</div>
      <div class="slot-value">Seçim yok</div>
    </div>
  </div>
</div>

<div class="bottom-row">
  <div class="box half">
    <div class="panel-header">
      <h2 style="margin:0;">Kullanıcı Mesajları</h2>
      <button class="mini-btn" onclick="openPanelOverlay('user')">⤢</button>
    </div>
    <div id="userlogs" class="userlogs"></div>
    <div id="summaryBox" class="summary"></div>
  </div>

  <div class="box half">
    <div class="panel-header">
      <h2 style="margin:0;">Tüm Loglar</h2>
```

```
          <button class="mini-btn" onclick="openPanelOverlay('log')">⤢</button>
        </div>
        <div id="fulllogs" class="logs"></div>
      </div>
    </div>
  </div>

</div>

<div id="imgModal" class="modal">
  <div class="modal-content">
    <div class="modal-header">
      <h3>Yerleştirme Sonucu</h3>
      <button class="close-btn" onclick="closePopup()">✕</button>
    </div>
    <img id="resultImg" src="/debug-image?ts=0" alt="debug image">
  </div>
</div>

<div id="panelOverlay" class="panel-overlay">
  <div class="panel-overlay-content">
    <div class="modal-header">
      <h3 id="overlayTitle">Panel</h3>
      <button class="close-btn" onclick="closePanelOverlay()">✕</button>
    </div>
    <div class="panel-overlay-body" id="overlayBody"></div>
  </div>
```

```
</div>

<script>
const ALL_RES = ["223", "331", "1001", "332"];
let prevRunning = false;
let overlayMode = null;

function setOptions(selectId, values) {
  const el = document.getElementById(selectId);
  const current = el.value;
  el.innerHTML = "";

  const noneOpt = document.createElement("option");
  noneOpt.value = "";
  noneOpt.textContent = "None";
  el.appendChild(noneOpt);

  values.forEach(v => {
    const opt = document.createElement("option");
    opt.value = v;
    opt.textContent = v;
    el.appendChild(opt);
  });

  if (["", ...values].includes(current)) {
    el.value = current;
  } else {
```

```
    el.value = "";
  }
}

function renderUserMessages(messages, targetEl) {
  targetEl.innerHTML = "";
  messages.forEach(msg => {
    const div = document.createElement("div");
    div.className = "user-msg";
    div.textContent = msg;
    targetEl.appendChild(div);
  });
}

function setSlot(slotId, valueText) {
  const slot = document.getElementById(slotId);
  const valueDiv = slot.querySelector(".slot-value");

  if (!valueText) {
    valueDiv.innerText = "Seçim yok";
    slot.classList.add("hint");
    slot.classList.remove("active");
    return;
  }

  valueDiv.innerText = valueText;
  slot.classList.remove("hint");
```

```
  slot.classList.add("active");
}

function resetFormToDefault() {
  document.getElementById("mode").value = "1";

  document.getElementById("d1_diode").value = "";
  document.getElementById("r1_res").value = "";
  document.getElementById("r2_res").value = "";

  document.getElementById("m2_d1").value = "";
  document.getElementById("m2_d2").value = "";
  document.getElementById("m2_r1").value = "";
  document.getElementById("m2_r2").value = "";

  updateModeFields();

  setSlot("slot_D1", "");
  setSlot("slot_D2", "");
  setSlot("slot_R1", "");
  setSlot("slot_R2", "");
}

function syncUniqueMode1() {
  const r1 = document.getElementById("r1_res").value;
  const r2 = document.getElementById("r2_res").value;
```

```
  setOptions("r1_res", ALL_RES.filter(v => v !== r2 || r1 === v));
  setOptions("r2_res", ALL_RES.filter(v => v !== r1 || r2 === v));

  document.getElementById("r1_res").value = r1 && (r1 !== r2 || r2 === "") ? r1 : (r1 === r2
? "" : r1);
  document.getElementById("r2_res").value = r2 && (r2 !== r1 || r1 === "") ? r2 : (r1 === r2
? "" : r2);
}

function syncUniqueMode2() {
  const ids = ["m2_d1", "m2_d2", "m2_r1", "m2_r2"];
  const selected = {};
  ids.forEach(id => selected[id] = document.getElementById(id).value);

  ids.forEach(id => {
    const usedByOthers = ids
      .filter(other => other !== id)
      .map(other => selected[other])
      .filter(v => v !== "");

    const current = selected[id];
    const allowed = ALL_RES.filter(v => !usedByOthers.includes(v) || v === current);

    setOptions(id, allowed);

    if (current && allowed.includes(current)) {
      document.getElementById(id).value = current;
    } else {
```

```
      document.getElementById(id).value = "";
    }
  });
}

function updateModeFields() {
  const mode = document.getElementById("mode").value;
  document.getElementById("mode1-fields").style.display = mode === "1" ? "block" :
"none";
  document.getElementById("mode2-fields").style.display = mode === "2" ? "block" :
"none";

  const mode1Warning = document.getElementById("mode1-warning");
  if (mode1Warning) {
    mode1Warning.style.display = mode === "1" ? "block" : "none";
  }

  if (mode === "1") {
    syncUniqueMode1();
  } else {
    syncUniqueMode2();
  }

  updatePcbLabels();
}

function updatePcbLabels() {
  const mode = document.getElementById("mode").value;
```

```
  if (mode === "1") {
    syncUniqueMode1();

    const d1 = document.getElementById("d1_diode").value;
    const d2 = d1 === "S3M" ? "SK510C" : (d1 === "SK510C" ? "S3M" : "");
    const r1 = document.getElementById("r1_res").value;
    const r2 = document.getElementById("r2_res").value;

    setSlot("slot_D1", d1);
    setSlot("slot_D2", d2);
    setSlot("slot_R1", r1);
    setSlot("slot_R2", r2);
  } else {
    syncUniqueMode2();

    setSlot("slot_D1", document.getElementById("m2_d1").value);
    setSlot("slot_D2", document.getElementById("m2_d2").value);
    setSlot("slot_R1", document.getElementById("m2_r1").value);
    setSlot("slot_R2", document.getElementById("m2_r2").value);
  }
}

function validateBeforeStart() {
  const mode = document.getElementById("mode").value;

  if (mode === "1") {
```

```
  const d1 = document.getElementById("d1_diode").value;
  const r1 = document.getElementById("r1_res").value;
  const r2 = document.getElementById("r2_res").value;

  if (!d1 || !r1 || !r2) {
    alert("Mod 1 için tüm seçimleri yapmalısın.");
    return false;
  }

  if (r1 === r2) {
    alert("Aynı direnç değeri iki slotta seçilemez.");
    return false;
  }
} else {
  const vals = [
    document.getElementById("m2_d1").value,
    document.getElementById("m2_d2").value,
    document.getElementById("m2_r1").value,
    document.getElementById("m2_r2").value
  ];

  if (vals.some(v => !v)) {
    alert("Mod 2 için dört slotun da seçilmesi gerekiyor.");
    return false;
  }

  const uniqueVals = new Set(vals);
```

```
    if (uniqueVals.size !== 4) {
      alert("Mod 2 için aynı direnç değeri tekrar seçilemez.");
      return false;
    }
  }

  return true;
}

async function startSystem() {
  if (!validateBeforeStart()) {
    return;
  }

  const mode = document.getElementById("mode").value;
  let payload = { mode: mode };

  if (mode === "1") {
    const d1 = document.getElementById("d1_diode").value;
    const d2 = d1 === "S3M" ? "SK510C" : "S3M";

    payload.diode_slot_map = {};
    payload.diode_slot_map[d1] = "D1";
    payload.diode_slot_map[d2] = "D2";

    payload.resistor_map_mode1 = {
      [document.getElementById("r1_res").value]: "R1",
```

```
      [document.getElementById("r2_res").value]: "R2"
    };
  } else {
    payload.resistor_map_mode2 = {
      [document.getElementById("m2_d1").value]: "D1",
      [document.getElementById("m2_d2").value]: "D2",
      [document.getElementById("m2_r1").value]: "R1",
      [document.getElementById("m2_r2").value]: "R2"
    };
  }

  const res = await fetch("/start", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(payload)
  });
  const data = await res.json();
  alert(data.message);
}

async function closePopup() {
  document.getElementById("imgModal").style.display = "none";
  try {
    await fetch("/close-popup", { method: "POST" });
  } catch (e) {
    console.error("Popup kapatma hatası:", e);
  }
```

```
}

function openPanelOverlay(type) {
  overlayMode = type;
  const overlay = document.getElementById("panelOverlay");
  const body = document.getElementById("overlayBody");
  const title = document.getElementById("overlayTitle");

  body.innerHTML = "";

  if (type === "user") {
    title.textContent = "Kullanıcı Mesajları";
    const div = document.createElement("div");
    div.className = "userlogs";
    div.id = "overlayUserLogs";
    body.appendChild(div);
  } else {
    title.textContent = "Tüm Loglar";
    const div = document.createElement("div");
    div.className = "logs";
    div.id = "overlayFullLogs";
    body.appendChild(div);
  }

  overlay.style.display = "flex";
}
```

```
function closePanelOverlay() {
  document.getElementById("panelOverlay").style.display = "none";
  document.getElementById("overlayBody").innerHTML = "";
  overlayMode = null;
}

async function pollStatus() {
  const res = await fetch("/status");
  const data = await res.json();

  document.getElementById("fulllogs").innerText = data.full_logs.join("\\n");
  renderUserMessages(data.user_logs, document.getElementById("userlogs"));

  const full = document.getElementById("fulllogs");
  full.scrollTop = full.scrollHeight;

  const user = document.getElementById("userlogs");
  user.scrollTop = user.scrollHeight;

  if (overlayMode === "user") {
    const overlayUser = document.getElementById("overlayUserLogs");
    if (overlayUser) {
      renderUserMessages(data.user_logs, overlayUser);
      overlayUser.scrollTop = overlayUser.scrollHeight;
    }
  }
```

```
  if (overlayMode === "log") {
    const overlayFull = document.getElementById("overlayFullLogs");
    if (overlayFull) {
      overlayFull.innerText = data.full_logs.join("\\n");
      overlayFull.scrollTop = overlayFull.scrollHeight;
    }
  }

  const runState = document.getElementById("runState");
  if (data.running) {
    runState.innerText = "Durum: Çalışıyor";
    runState.className = "statusline status-running";
  } else {
    runState.innerText = "Durum: Beklemede";
    runState.className = "statusline status-idle";
  }

  document.getElementById("startBtn").disabled = data.running;

  if (data.result_summary) {
    document.getElementById("summaryBox").innerText =
JSON.stringify(data.result_summary, null, 2);
  } else {
    document.getElementById("summaryBox").innerText = "";
  }

  if (data.popup_open) {
```

```
    document.getElementById("imgModal").style.display = "flex";
    document.getElementById("resultImg").src = "/debug-image?ts=" + Date.now();
  }

  if (prevRunning === true && data.running === false) {
    resetFormToDefault();
  }

  prevRunning = data.running;

  setTimeout(pollStatus, 1000);
}

document.addEventListener("change", function() {
  updatePcbLabels();
});

document.addEventListener("keydown", function(e) {
  const modal = document.getElementById("imgModal");
  const overlay = document.getElementById("panelOverlay");

  if (e.key === "Escape") {
    if (modal.style.display === "flex") {
      closePopup();
      return;
    }
    if (overlay.style.display === "flex") {
```

```
      closePanelOverlay();
    }
  }
});

window.addEventListener("click", function(e) {
  const modal = document.getElementById("imgModal");
  const overlay = document.getElementById("panelOverlay");

  if (e.target === modal) {
    closePopup();
  }
  if (e.target === overlay) {
    closePanelOverlay();
  }
});

updateModeFields();
pollStatus();
</script>
</body>
</html>
"""

@app.route("/")
def index():
    return render_template_string(HTML_PAGE)
```

```
@app.route("/start", methods=["POST"])
def start():
    data = request.get_json(force=True)

    with LOCK:
        if STATE["running"]:
            return jsonify({"ok": False, "message": "Sistem zaten çalışıyor."})

    try:
        validate_config(data)
    except Exception as e:
        return jsonify({"ok": False, "message": f"Konfigürasyon hatası: {e}"})

    reset_state_for_new_run(data)
    t = threading.Thread(target=run_cycle_background, args=(data,), daemon=True)
    t.start()

    return jsonify({"ok": True, "message": "Sistem başlatıldı."})

@app.route("/status")
def status():
    with LOCK:
        return jsonify({
            "running": STATE["running"],
            "full_logs": STATE["full_logs"],
            "user_logs": STATE["user_logs"],
```

```
        "popup_open": STATE["popup_open"],
        "result_image_ready": STATE["result_image_ready"],
        "result_summary": STATE["result_summary"],
        "current_config": STATE["current_config"]
    })

@app.route("/debug-image")
def debug_image():
    if os.path.exists(ms.PLACEMENT_DEBUG_IMG):
        return send_file(ms.PLACEMENT_DEBUG_IMG)
    return "Resim yok", 404

@app.route("/close-popup", methods=["POST"])
def close_popup():
    with LOCK:
        STATE["popup_open"] = False
    return jsonify({"ok": True})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)
```